paul graham on startups

essays, volume five

13 essays, 38,921 words. all of them are at paulgraham.com, free, as they should be.
this is one reader's copy, printed at home to be read away from a screen, with a pencil.
a printed book of essays
github.com/workinprogmess/a-printed-book-of-essays

contents

01	how to get startup ideas	5
02	organic startup ideas	25
03	schlep blindness	28
04	frighteningly ambitious startup ideas	31
05	do things that don't scale	42
06	startup = growth	55
07	before the startup	70
08	startups in 13 sentences	83
09	the reddits	88
10	how to start google	92
11	founder mode	100
12	how universities should prepare founders	104

01

how to get startup ideas

paulgraham.com/startupideas.html

november 2012

the way to get startup ideas is not to try to think of startup ideas. it's to look for problems, preferably problems you have yourself.

the very best startup ideas tend to have three things in common: they're something the founders themselves want, that they themselves can build, and that few others realize are worth doing. microsoft, apple, yahoo, google, and facebook all began this way.

problems

why is it so important to work on a problem you have? among other things, it ensures the problem really exists. it sounds obvious to say you should only work on problems that exist. and yet by far the most common mistake startups make is to solve problems no one has.

i made it myself. in 1995 i started a company to put art galleries online. but galleries didn't want to be online. it's not how the art business works. so why did i spend 6 months working on this stupid idea? because i didn't pay attention to users. i invented a model of the world that didn't correspond to reality, and worked from that. i didn't notice my model was wrong until i tried to convince users to pay for what we'd built. even then i took embarrassingly long to catch on. i was attached to my model of the world, and i'd spent a lot of time on the software. they had to want it!

why do so many founders build things no one wants? because they begin by trying to think of startup ideas. that m.o. is doubly dangerous: it doesn't

merely yield few good ideas; it yields bad ideas that sound plausible enough to fool you into working on them.

at yc we call these "made-up" or "sitcom" startup ideas. imagine one of the characters on a tv show was starting a startup. the writers would have to invent something for it to do. but coming up with good startup ideas is hard. it's not something you can do for the asking. so (unless they got amazingly lucky) the writers would come up with an idea that sounded plausible, but was actually bad.

for example, a social network for pet owners. it doesn't sound obviously mistaken. millions of people have pets. often they care a lot about their pets and spend a lot of money on them. surely many of these people would like a site where they could talk to other pet owners. not all of them perhaps, but if just 2 or 3 percent were regular visitors, you could have millions of users. you could serve them targeted offers, and maybe charge for premium features. [1]

the danger of an idea like this is that when you run it by your friends with pets, they don't say "i would *never* use this." they say "yeah, maybe i could see using something like that." even when the startup launches, it will sound plausible to a lot of people. they don't want to use it themselves, at least not right now, but they could imagine other people wanting it. sum that reaction across the entire population, and you have zero users. [2]

well

when a startup launches, there have to be at least some users who really need what they're making — not just people who could see themselves using it one day, but who want it urgently. usually this initial group of users is small, for the simple reason that if there were something that large numbers of people urgently needed and that could be built with the amount of effort a startup usually puts into a version one, it would probably already exist. which means you have to compromise on one dimension: you can either build something a large number of people want a small amount, or something a small number of people want a large amount. choose the latt-

er. not all ideas of that type are good startup ideas, but nearly all good startup ideas are of that type.

imagine a graph whose x axis represents all the people who might want what you're making and whose y axis represents how much they want it. if you invert the scale on the y axis, you can envision companies as holes. google is an immense crater: hundreds of millions of people use it, and they need it a lot. a startup just starting out can't expect to excavate that much volume. so you have two choices about the shape of hole you start with. you can either dig a hole that's broad but shallow, or one that's narrow and deep, like a well.

made-up startup ideas are usually of the first type. lots of people are mildly interested in a social network for pet owners.

nearly all good startup ideas are of the second type. microsoft was a well when they made altair basic. there were only a couple thousand altair owners, but without this software they were programming in machine language. thirty years later facebook had the same shape. their first site was exclusively for harvard students, of which there are only a few thousand, but those few thousand users wanted it a lot.

when you have an idea for a startup, ask yourself: who wants this right now? who wants this so much that they'll use it even when it's a crappy version one made by a two-person startup they've never heard of? if you can't answer that, the idea is probably bad. [3]

you don't need the narrowness of the well per se. it's depth you need; you get narrowness as a byproduct of optimizing for depth (and speed). but you almost always do get it. in practice the link between depth and narrowness is so strong that it's a good sign when you know that an idea will appeal strongly to a specific group or type of user.

but while demand shaped like a well is almost a necessary condition for a good startup idea, it's not a sufficient one. if mark zuckerberg had built something that could only ever have appealed to harvard students, it would not have been a good startup idea. facebook was a good idea because it started with a small market there was a fast path out of. colleges

are similar enough that if you build a facebook that works at harvard, it will work at any college. so you spread rapidly through all the colleges. once you have all the college students, you get everyone else simply by letting them in.

similarly for microsoft: basic for the altair; basic for other machines; other languages besides basic; operating systems; applications; ipo.

self

how do you tell whether there's a path out of an idea? how do you tell whether something is the germ of a giant company, or just a niche product? often you can't. the founders of airbnb didn't realize at first how big a market they were tapping. initially they had a much narrower idea. they were going to let hosts rent out space on their floors during conventions. they didn't foresee the expansion of this idea; it forced itself upon them gradually. all they knew at first is that they were onto something. that's probably as much as bill gates or mark zuckerberg knew at first.

occasionally it's obvious from the beginning when there's a path out of the initial niche. and sometimes i can see a path that's not immediately obvious; that's one of our specialties at yc. but there are limits to how well this can be done, no matter how much experience you have. the most important thing to understand about paths out of the initial idea is the meta-fact that these are hard to see.

so if you can't predict whether there's a path out of an idea, how do you choose between ideas? the truth is disappointing but interesting: if you're the right sort of person, you have the right sort of hunches. if you're at the leading edge of a field that's changing fast, when you have a hunch that something is worth doing, you're more likely to be right.

in *zen and the art of motorcycle maintenance*, robert pirsig says:

> you want to know how to paint a perfect painting? it's easy. make yourself perfect and then just paint naturally.

i've wondered about that passage since i read it in high school. i'm not sure how useful his advice is for painting specifically, but it fits this situation

well. empirically, the way to have good startup ideas is to become the sort of person who has them.
being at the leading edge of a field doesn't mean you have to be one of the people pushing it forward. you can also be at the leading edge as a user. it was not so much because he was a programmer that facebook seemed a good idea to mark zuckerberg as because he used computers so much. if you'd asked most 40 year olds in 2004 whether they'd like to publish their lives semi-publicly on the internet, they'd have been horrified at the idea. but mark already lived online; to him it seemed natural.
paul buchheit says that people at the leading edge of a rapidly changing field "live in the future." combine that with pirsig and you get:

> live in the future, then build what's missing.

that describes the way many if not most of the biggest startups got started. neither apple nor yahoo nor google nor facebook were even supposed to be companies at first. they grew out of things their founders built because there seemed a gap in the world.
if you look at the way successful founders have had their ideas, it's generally the result of some external stimulus hitting a prepared mind. bill gates and paul allen hear about the altair and think "i bet we could write a basic interpreter for it." drew houston realizes he's forgotten his usb stick and thinks "i really need to make my files live online." lots of people heard about the altair. lots forgot usb sticks. the reason those stimuli caused those founders to start companies was that their experiences had prepared them to notice the opportunities they represented.
the verb you want to be using with respect to startup ideas is not "think up" but "notice." at yc we call ideas that grow naturally out of the founders' own experiences "organic" startup ideas. the most successful startups almost all begin this way.
that may not have been what you wanted to hear. you may have expected recipes for coming up with startup ideas, and instead i'm telling you that the key is to have a mind that's prepared in the right way. but disappointi-

ng though it may be, this is the truth. and it is a recipe of a sort, just one that in the worst case takes a year rather than a weekend.

if you're not at the leading edge of some rapidly changing field, you can get to one. for example, anyone reasonably smart can probably get to an edge of programming (e.g. building mobile apps) in a year. since a successful startup will consume at least 3-5 years of your life, a year's preparation would be a reasonable investment. especially if you're also looking for a cofounder. [4]

you don't have to learn programming to be at the leading edge of a domain that's changing fast. other domains change fast. but while learning to hack is not necessary, it is for the foreseeable future sufficient. as marc andreessen put it, software is eating the world, and this trend has decades left to run.

knowing how to hack also means that when you have ideas, you'll be able to implement them. that's not absolutely necessary (jeff bezos couldn't) but it's an advantage. it's a big advantage, when you're considering an idea like putting a college facebook online, if instead of merely thinking "that's an interesting idea," you can think instead "that's an interesting idea. i'll try building an initial version tonight." it's even better when you're both a programmer and the target user, because then the cycle of generating new versions and testing them on users can happen inside one head.

noticing

once you're living in the future in some respect, the way to notice startup ideas is to look for things that seem to be missing. if you're really at the leading edge of a rapidly changing field, there will be things that are obviously missing. what won't be obvious is that they're startup ideas. so if you want to find startup ideas, don't merely turn on the filter "what's missing?" also turn off every other filter, particularly "could this be a big company?" there's plenty of time to apply that test later. but if you're thinking about that initially, it may not only filter out lots of good ideas, but also cause you to focus on bad ones.

most things that are missing will take some time to see. you almost have to trick yourself into seeing the ideas around you.

but you *know* the ideas are out there. this is not one of those problems where there might not be an answer. it's impossibly unlikely that this is the exact moment when technological progress stops. you can be sure people are going to build things in the next few years that will make you think "what did i do before x?"

and when these problems get solved, they will probably seem flamingly obvious in retrospect. what you need to do is turn off the filters that usually prevent you from seeing them. the most powerful is simply taking the current state of the world for granted. even the most radically open-minded of us mostly do that. you couldn't get from your bed to the front door if you stopped to question everything.

but if you're looking for startup ideas you can sacrifice some of the efficiency of taking the status quo for granted and start to question things. why is your inbox overflowing? because you get a lot of email, or because it's hard to get email out of your inbox? why do you get so much email? what problems are people trying to solve by sending you email? are there better ways to solve them? and why is it hard to get emails out of your inbox? why do you keep emails around after you've read them? is an inbox the optimal tool for that?

pay particular attention to things that chafe you. the advantage of taking the status quo for granted is not just that it makes life (locally) more efficient, but also that it makes life more tolerable. if you knew about all the things we'll get in the next 50 years but don't have yet, you'd find present day life pretty constraining, just as someone from the present would if they were sent back 50 years in a time machine. when something annoys you, it could be because you're living in the future.

when you find the right sort of problem, you should probably be able to describe it as *obvious*, at least to you. when we started viaweb, all the online stores were built by hand, by web designers making individual html

pages. it was obvious to us as programmers that these sites would have to be generated by software. [5]

which means, strangely enough, that coming up with startup ideas is a question of seeing the obvious. that suggests how weird this process is: you're trying to see things that are obvious, and yet that you hadn't seen. since what you need to do here is loosen up your own mind, it may be best not to make too much of a direct frontal attack on the problem — i.e. to sit down and try to think of ideas. the best plan may be just to keep a background process running, looking for things that seem to be missing. work on hard problems, driven mainly by curiosity, but have a second self watching over your shoulder, taking note of gaps and anomalies. [6]

give yourself some time. you have a lot of control over the rate at which you turn yours into a prepared mind, but you have less control over the stimuli that spark ideas when they hit it. if bill gates and paul allen had constrained themselves to come up with a startup idea in one month, what if they'd chosen a month before the altair appeared? they probably would have worked on a less promising idea. drew houston did work on a less promising idea before dropbox: an sat prep startup. but dropbox was a much better idea, both in the absolute sense and also as a match for his skills. [7]

a good way to trick yourself into noticing ideas is to work on projects that seem like they'd be cool. if you do that, you'll naturally tend to build things that are missing. it wouldn't seem as interesting to build something that already existed.

just as trying to think up startup ideas tends to produce bad ones, working on things that could be dismissed as "toys" often produces good ones. when something is described as a toy, that means it has everything an idea needs except being important. it's cool; users love it; it just doesn't matter. but if you're living in the future and you build something cool that users love, it may matter more than outsiders think. microcomputers seemed like toys when apple and microsoft started working on them. i'm old enough to remember that era; the usual term for people with their own microcomputers

was "hobbyists." backrub seemed like an inconsequential science project. the facebook was just a way for undergrads to stalk one another.
at yc we're excited when we meet startups working on things that we could imagine know-it-alls on forums dismissing as toys. to us that's positive evidence an idea is good.
if you can afford to take a long view (and arguably you can't afford not to), you can turn "live in the future and build what's missing" into something even better:

> live in the future and build what seems interesting.

school
that's what i'd advise college students to do, rather than trying to learn about "entrepreneurship." "entrepreneurship" is something you learn best by doing it. the examples of the most successful founders make that clear. what you should be spending your time on in college is ratcheting yourself into the future. college is an incomparable opportunity to do that. what a waste to sacrifice an opportunity to solve the hard part of starting a startup — becoming the sort of person who can have organic startup ideas — by spending time learning about the easy part. especially since you won't even really learn about it, any more than you'd learn about sex in a class. all you'll learn is the words for things.
the clash of domains is a particularly fruitful source of ideas. if you know a lot about programming and you start learning about some other field, you'll probably see problems that software could solve. in fact, you're doubly likely to find good problems in another domain: (a) the inhabitants of that domain are not as likely as software people to have already solved their problems with software, and (b) since you come into the new domain totally ignorant, you don't even know what the status quo is to take it for granted.
so if you're a cs major and you want to start a startup, instead of taking a class on entrepreneurship you're better off taking a class on, say, genetics.

er jobs at computer hardware or software companies. but if you want to find startup ideas, you might do better to get a summer job in some unrelated field. [8]

or don't take any extra classes, and just build things. it's no coincidence that microsoft and facebook both got started in january. at harvard that is (or was) reading period, when students have no classes to attend because they're supposed to be studying for finals. [9]

but don't feel like you have to build things that will become startups. that's premature optimization. just build things. preferably with other students. it's not just the classes that make a university such a good place to crank oneself into the future. you're also surrounded by other people trying to do the same thing. if you work together with them on projects, you'll end up producing not just organic ideas, but organic ideas with organic founding teams — and that, empirically, is the best combination.

beware of research. if an undergrad writes something all his friends start using, it's quite likely to represent a good startup idea. whereas a phd dissertation is extremely unlikely to. for some reason, the more a project has to count as research, the less likely it is to be something that could be turned into a startup. [10] i think the reason is that the subset of ideas that count as research is so narrow that it's unlikely that a project that satisfied that constraint would also satisfy the orthogonal constraint of solving users' problems. whereas when students (or professors) build something as a side-project, they automatically gravitate toward solving users' problems — perhaps even with an additional energy that comes from being freed from the constraints of research.

competition

because a good idea should seem obvious, when you have one you'll tend to feel that you're late. don't let that deter you. worrying that you're late is one of the signs of a good idea. ten minutes of searching the web will usually settle the question. even if you find someone else working on the same thing, you're probably not too late. it's exceptionally rare for startups to be killed by competitors — so rare that you can almost discount the possibilit-

y. so unless you discover a competitor with the sort of lock-in that would prevent users from choosing you, don't discard the idea.

if you're uncertain, ask users. the question of whether you're too late is subsumed by the question of whether anyone urgently needs what you plan to make. if you have something that no competitor does and that some subset of users urgently need, you have a beachhead. [11]

the question then is whether that beachhead is big enough. or more importantly, who's in it: if the beachhead consists of people doing something lots more people will be doing in the future, then it's probably big enough no matter how small it is. for example, if you're building something differentiated from competitors by the fact that it works on phones, but it only works on the newest phones, that's probably a big enough beachhead.

err on the side of doing things where you'll face competitors. inexperienced founders usually give competitors more credit than they deserve. whether you succeed depends far more on you than on your competitors. so better a good idea with competitors than a bad one without.

you don't need to worry about entering a "crowded market" so long as you have a thesis about what everyone else in it is overlooking. in fact that's a very promising starting point. google was that type of idea. your thesis has to be more precise than "we're going to make an x that doesn't suck" though. you have to be able to phrase it in terms of something the incumbents are overlooking. best of all is when you can say that they didn't have the courage of their convictions, and that your plan is what they'd have done if they'd followed through on their own insights. google was that type of idea too. the search engines that preceded them shied away from the most radical implications of what they were doing — particularly that the better a job they did, the faster users would leave.

a crowded market is actually a good sign, because it means both that there's demand and that none of the existing solutions are good enough. a startup can't hope to enter a market that's obviously big and yet in which they have no competitors. so any startup that succeeds is either going to be entering a market with existing competitors, but armed with some secret

weapon that will get them all the users (like google), or entering a market that looks small but which will turn out to be big (like microsoft). [12]

filters

there are two more filters you'll need to turn off if you want to notice startup ideas: the unsexy filter and the schlep filter.

most programmers wish they could start a startup by just writing some brilliant code, pushing it to a server, and having users pay them lots of money. they'd prefer not to deal with tedious problems or get involved in messy ways with the real world. which is a reasonable preference, because such things slow you down. but this preference is so widespread that the space of convenient startup ideas has been stripped pretty clean. if you let your mind wander a few blocks down the street to the messy, tedious ideas, you'll find valuable ones just sitting there waiting to be implemented.

the schlep filter is so dangerous that i wrote a separate essay about the condition it induces, which i called schlep blindness. i gave stripe as an example of a startup that benefited from turning off this filter, and a pretty striking example it is. thousands of programmers were in a position to see this idea; thousands of programmers knew how painful it was to process payments before stripe. but when they looked for startup ideas they didn't see this one, because unconsciously they shrank from having to deal with payments. and dealing with payments is a schlep for stripe, but not an intolerable one. in fact they might have had net less pain; because the fear of dealing with payments kept most people away from this idea, stripe has had comparatively smooth sailing in other areas that are sometimes painful, like user acquisition. they didn't have to try very hard to make themselves heard by users, because users were desperately waiting for what they were building.

the unsexy filter is similar to the schlep filter, except it keeps you from working on problems you despise rather than ones you fear. we overcame this one to work on viaweb. there were interesting things about the archit-

ecture of our software, but we weren't interested in ecommerce per se. we could see the problem was one that needed to be solved though.

turning off the schlep filter is more important than turning off the unsexy filter, because the schlep filter is more likely to be an illusion. and even to the degree it isn't, it's a worse form of self-indulgence. starting a successful startup is going to be fairly laborious no matter what. even if the product doesn't entail a lot of schleps, you'll still have plenty dealing with investors, hiring and firing people, and so on. so if there's some idea you think would be cool but you're kept away from by fear of the schleps involved, don't worry: any sufficiently good idea will have as many.

the unsexy filter, while still a source of error, is not as entirely useless as the schlep filter. if you're at the leading edge of a field that's changing rapidly, your ideas about what's sexy will be somewhat correlated with what's valuable in practice. particularly as you get older and more experienced. plus if you find an idea sexy, you'll work on it more enthusiastically. [13]

recipes

while the best way to discover startup ideas is to become the sort of person who has them and then build whatever interests you, sometimes you don't have that luxury. sometimes you need an idea now. for example, if you're working on a startup and your initial idea turns out to be bad.

for the rest of this essay i'll talk about tricks for coming up with startup ideas on demand. although empirically you're better off using the organic strategy, you could succeed this way. you just have to be more disciplined. when you use the organic method, you don't even notice an idea unless it's evidence that something is truly missing. but when you make a conscious effort to think of startup ideas, you have to replace this natural constraint with self-discipline. you'll see a lot more ideas, most of them bad, so you need to be able to filter them.

one of the biggest dangers of not using the organic method is the example of the organic method. organic ideas feel like inspirations. there are a lot of stories about successful startups that began when the founders had what

seemed a crazy idea but "just knew" it was promising. when you feel that about an idea you've had while trying to come up with startup ideas, you're probably mistaken.

when searching for ideas, look in areas where you have some expertise. if you're a database expert, don't build a chat app for teenagers (unless you're also a teenager). maybe it's a good idea, but you can't trust your judgment about that, so ignore it. there have to be other ideas that involve databases, and whose quality you can judge. do you find it hard to come up with good ideas involving databases? that's because your expertise raises your standards. your ideas about chat apps are just as bad, but you're giving yourself a dunning-kruger pass in that domain.

the place to start looking for ideas is things you need. there *must* be things you need. [14]

one good trick is to ask yourself whether in your previous job you ever found yourself saying "why doesn't someone make x? if someone made x we'd buy it in a second." if you can think of any x people said that about, you probably have an idea. you know there's demand, and people don't say that about things that are impossible to build.

more generally, try asking yourself whether there's something unusual about you that makes your needs different from most other people's. you're probably not the only one. it's especially good if you're different in a way people will increasingly be.

if you're changing ideas, one unusual thing about you is the idea you'd previously been working on. did you discover any needs while working on it? several well-known startups began this way. hotmail began as something its founders wrote to talk about their previous startup idea while they were working at their day jobs. [15]

a particularly promising way to be unusual is to be young. some of the most valuable new ideas take root first among people in their teens and early twenties. and while young founders are at a disadvantage in some respects, they're the only ones who really understand their peers. it would have been very hard for someone who wasn't a college student to start

facebook. so if you're a young founder (under 23 say), are there things you and your friends would like to do that current technology won't let you? the next best thing to an unmet need of your own is an unmet need of someone else. try talking to everyone you can about the gaps they find in the world. what's missing? what would they like to do that they can't? what's tedious or annoying, particularly in their work? let the conversation get general; don't be trying too hard to find startup ideas. you're just looking for something to spark a thought. maybe you'll notice a problem they didn't consciously realize they had, because you know how to solve it.

when you find an unmet need that isn't your own, it may be somewhat blurry at first. the person who needs something may not know exactly what they need. in that case i often recommend that founders act like consultants — that they do what they'd do if they'd been retained to solve the problems of this one user. people's problems are similar enough that nearly all the code you write this way will be reusable, and whatever isn't will be a small price to start out certain that you've reached the bottom of the well. [16]

one way to ensure you do a good job solving other people's problems is to make them your own. when rajat suri of e la carte decided to write software for restaurants, he got a job as a waiter to learn how restaurants worked. that may seem like taking things to extremes, but startups are extreme. we love it when founders do such things.

in fact, one strategy i recommend to people who need a new idea is not merely to turn off their schlep and unsexy filters, but to seek out ideas that are unsexy or involve schleps. don't try to start twitter. those ideas are so rare that you can't find them by looking for them. make something unsexy that people will pay you for.

a good trick for bypassing the schlep and to some extent the unsexy filter is to ask what you wish someone else would build, so that you could use it. what would you pay for right now?

since startups often garbage-collect broken companies and industries, it can be a good trick to look for those that are dying, or deserve to, and try

to imagine what kind of company would profit from their demise. for example, journalism is in free fall at the moment. but there may still be money to be made from something like journalism. what sort of company might cause people in the future to say "this replaced journalism" on some axis? but imagine asking that in the future, not now. when one company or industry replaces another, it usually comes in from the side. so don't look for a replacement for x; look for something that people will later say turned out to be a replacement for x. and be imaginative about the axis along which the replacement occurs. traditional journalism, for example, is a way for readers to get information and to kill time, a way for writers to make money and to get attention, and a vehicle for several different types of advertising. it could be replaced on any of these axes (it has already started to be on most).

when startups consume incumbents, they usually start by serving some small but important market that the big players ignore. it's particularly good if there's an admixture of disdain in the big players' attitude, because that often misleads them. for example, after steve wozniak built the computer that became the apple i, he felt obliged to give his then-employer hewlett-packard the option to produce it. fortunately for him, they turned it down, and one of the reasons they did was that it used a tv for a monitor, which seemed intolerably déclassé to a high-end hardware company like hp was at the time. [17]

are there groups of scruffy but sophisticated users like the early microcomputer "hobbyists" that are currently being ignored by the big players? a startup with its sights set on bigger things can often capture a small market easily by expending an effort that wouldn't be justified by that market alone.

similarly, since the most successful startups generally ride some wave bigger than themselves, it could be a good trick to look for waves and ask how one could benefit from them. the prices of gene sequencing and 3d printing are both experiencing moore's law-like declines. what new things will we

be able to do in the new world we'll have in a few years? what are we unconsciously ruling out as impossible that will soon be possible?

organic

but talking about looking explicitly for waves makes it clear that such recipes are plan b for getting startup ideas. looking for waves is essentially a way to simulate the organic method. if you're at the leading edge of some rapidly changing field, you don't have to look for waves; you are the wave. finding startup ideas is a subtle business, and that's why most people who try fail so miserably. it doesn't work well simply to try to think of startup ideas. if you do that, you get bad ones that sound dangerously plausible. the best approach is more indirect: if you have the right sort of background, good startup ideas will seem obvious to you. but even then, not immediately. it takes time to come across situations where you notice something missing. and often these gaps won't seem to be ideas for companies, just things that would be interesting to build. which is why it's good to have the time and the inclination to build things just because they're interesting.

live in the future and build what seems interesting. strange as it sounds, that's the real recipe.

---

notes

1 this form of bad idea has been around as long as the web. it was common in the 1990s, except then people who had it used to say they were going to create a portal for x instead of a social network for x. structurally the idea is stone soup: you post a sign saying "this is the place for people interested in x," and all those people show up and you make money from them. what lures founders into this sort of idea are statistics about the millions of people who might be interested in each type of x. what they forget is that any given person might have 20 affinities by this standard, and no one is going to visit 20 different communities regularly.

2 i'm not saying, incidentally, that i know for sure a social network for pet owners is a bad idea. i know it's a bad idea the way i know randomly generated dna would not produce a viable organism. the set of plausible sounding startup ideas is many times larger than the set of good ones, and many of the good ones don't even sound that plausible. so if all you know about a startup idea is that it sounds plausible, you have to assume it's bad.

3 more precisely, the users' need has to give them sufficient activation energy to start using whatever you make, which can vary a lot. for example, the activation energy for enterprise software sold through traditional channels is very high, so you'd have to be a *lot* better to get users to switch. whereas the activation energy required to switch to a new search engine is low. which in turn is why search engines are so much better than enterprise software.

4 this gets harder as you get older. while the space of ideas doesn't have dangerous local maxima, the space of careers does. there are fairly high walls between most of the paths people take through life, and the older you get, the higher the walls become.

5 it was also obvious to us that the web was going to be a big deal. few non-programmers grasped that in 1995, but the programmers had seen what guis had done for desktop computers.

6 maybe it would work to have this second self keep a journal, and each night to make a brief entry listing the gaps and anomalies you'd noticed that day. not startup ideas, just the raw gaps and anomalies.

7 sam altman points out that taking time to come up with an idea is not merely a better strategy in an absolute sense, but also like an undervalued stock in that so few founders do it.

there's comparatively little competition for the best ideas, because few founders are willing to put in the time required to notice them. whereas there is a great deal of competition for mediocre ideas, because when people make up startup ideas, they tend to make up the same ones.

8 for the computer hardware and software companies, summer jobs are the first phase of the recruiting funnel. but if you're good you can skip the first phase. if you're good you'll have no trouble getting hired by these companies when you graduate, regardless of how you spent your summers.

9 the empirical evidence suggests that if colleges want to help their students start startups, the best thing they can do is leave them alone in the right way.

10 i'm speaking here of it startups; in biotech things are different.

11 this is an instance of a more general rule: focus on users, not competitors. the most important information about competitors is what you learn via users anyway.

12 in practice most successful startups have elements of both. and you can describe each strategy in terms of the other by adjusting the boundaries of what you call the market. but it's useful to consider these two ideas separately.

13 i almost hesitate to raise that point though. startups are businesses; the point of a business is to make money; and with that additional constraint, you can't expect you'll be able to spend all your time working on what interests you most.

14 the need has to be a strong one. you can retroactively describe any made-up idea as something you need. but do you really need that recipe site or local event aggregator as much as drew houston needed dropbox, or brian chesky and joe gebbia needed airbnb?

quite often at yc i find myself asking founders "would you use this thing yourself, if you hadn't written it?" and you'd be surprised how often the answer is no.

15 paul buchheit points out that trying to sell something bad can be a source of better ideas:

"the best technique i've found for dealing with yc companies that have bad ideas is to tell them to go sell the product asap (before wasting time building it). not only do they learn that nobody wants what they are building, they very often come back with a real idea that they discovered in the process of trying to sell the bad idea."

16 here's a recipe that might produce the next facebook, if you're college students. if you have a connection to one of the more powerful sororities at your school, approach the queen bees thereof and offer to be their personal it consultants, building anything they could imagine needing in their social lives that didn't already exist. anything that got built this way would be very promising, because such users are not just the most demanding but also the perfect point to spread from.

i have no idea whether this would work.

17 and the reason it used a tv for a monitor is that steve wozniak started out by solving his own problems. he, like most of his peers, couldn't afford a monitor.

thanks to sam altman, mike arrington, paul buchheit, john collison, patrick collison, garry tan, and harj taggar for reading drafts of this, and marc andreessen, joe gebbia, reid hoffman, shel kaphan, mike moritz and kevin systrom for answering my questions about startup history.

02

organic startup ideas

paulgraham.com/organic.html

april 2010

the best way to come up with startup ideas is to ask yourself the question: what do you wish someone would make for you?

there are two types of startup ideas: those that grow organically out of your own life, and those that you decide, from afar, are going to be necessary to some class of users other than you. apple was the first type. apple happened because steve wozniak wanted a computer. unlike most people who wanted computers, he could design one, so he did. and since lots of other people wanted the same thing, apple was able to sell enough of them to get the company rolling. they still rely on this principle today, incidentally. the iphone is the phone steve jobs wants. [1]

our own startup, viaweb, was of the second type. we made software for building online stores. we didn't need this software ourselves. we weren't direct marketers. we didn't even know when we started that our users were called "direct marketers." but we were comparatively old when we started the company (i was 30 and robert morris was 29), so we'd seen enough to know users would need this type of software. [2]

there is no sharp line between the two types of ideas, but the most successful startups seem to be closer to the apple type than the viaweb type. when he was writing that first basic interpreter for the altair, bill gates was writing something he would use, as were larry and sergey when they wrote the first versions of google.

organic ideas are generally preferable to the made up kind, but particularly so when the founders are young. it takes experience to predict what other people will want. the worst ideas we see at y combinator are from young founders making things they think other people will want.

so if you want to start a startup and don't know yet what you're going to do, i'd encourage you to focus initially on organic ideas. what's missing or broken in your daily life? sometimes if you just ask that question you'll get immediate answers. it must have seemed obviously broken to bill gates that you could only program the altair in machine language.

you may need to stand outside yourself a bit to see brokenness, because you tend to get used to it and take it for granted. you can be sure it's there, though. there are always great ideas sitting right under our noses. in 2004 it was ridiculous that harvard undergrads were still using a facebook printed on paper. surely that sort of thing should have been online.

there are ideas that obvious lying around now. the reason you're overlooking them is the same reason you'd have overlooked the idea of building facebook in 2004: organic startup ideas usually don't seem like startup ideas at first. we know now that facebook was very successful, but put yourself back in 2004. putting undergraduates' profiles online wouldn't have seemed like much of a startup idea. and in fact, it wasn't initially a startup idea. when mark spoke at a yc dinner this winter he said he wasn't trying to start a company when he wrote the first version of facebook. it was just a project. so was the apple i when woz first started working on it. he didn't think he was starting a company. if these guys had thought they were starting companies, they might have been tempted to do something more "serious," and that would have been a mistake.

so if you want to come up with organic startup ideas, i'd encourage you to focus more on the idea part and less on the startup part. just fix things that seem broken, regardless of whether it seems like the problem is important enough to build a company on. if you keep pursuing such threads it would be hard not to end up making something of value to a lot of people, and when you do, surprise, you've got a company. [3]

don't be discouraged if what you produce initially is something other people dismiss as a toy. in fact, that's a good sign. that's probably why everyone else has been overlooking the idea. the first microcomputers were dismissed as toys. and the first planes, and the first cars. at this point, when someone comes to us with something that users like but that we could envision forum trolls dismissing as a toy, it makes us especially likely to invest.

while young founders are at a disadvantage when coming up with made-up ideas, they're the best source of organic ones, because they're at the forefront of technology. they use the latest stuff. they only just decided what to use, so why wouldn't they? and because they use the latest stuff, they're in a position to discover valuable types of fixable brokenness first. there's nothing more valuable than an unmet need that is just becoming fixable. if you find something broken that you can fix for a lot of people, you've found a gold mine. as with an actual gold mine, you still have to work hard to get the gold out of it. but at least you know where the seam is, and that's the hard part.

---

notes

1 this suggests a way to predict areas where apple will be weak: things steve jobs doesn't use. e.g. i doubt he is much into gaming.

2 in retrospect, we should have *become* direct marketers. if i were doing viaweb again, i'd open our own online store. if we had, we'd have understood users a lot better. i'd encourage anyone starting a startup to become one of its users, however unnatural it seems.

3 possible exception: it's hard to compete directly with open source software. you can build things for programmers, but there has to be some part you can charge for.

thanks to sam altman, trevor blackwell, and jessica livingston for reading drafts of this.

03

schlep blindness

paulgraham.com/schlep.html

january 2012

there are great startup ideas lying around unexploited right under our noses. one reason we don't see them is a phenomenon i call *schlep blindness*. schlep was originally a yiddish word but has passed into general use in the us. it means a tedious, unpleasant task.

no one likes schleps, but hackers especially dislike them. most hackers who start startups wish they could do it by just writing some clever software, putting it on a server somewhere, and watching the money roll in—without ever having to talk to users, or negotiate with other companies, or deal with other people's broken code. maybe that's possible, but i haven't seen it.

one of the many things we do at y combinator is teach hackers about the inevitability of schleps. no, you can't start a startup by just writing code. i remember going through this realization myself. there was a point in 1995 when i was still trying to convince myself i could start a company by just writing code. but i soon learned from experience that schleps are not merely inevitable, but pretty much what business consists of. a company is defined by the schleps it will undertake. and schleps should be dealt with the same way you'd deal with a cold swimming pool: just jump in. which is not to say you should seek out unpleasant work per se, but that you should never shrink from it if it's on the path to something great.

the most dangerous thing about our dislike of schleps is that much of it is unconscious. your unconscious won't even let you see ideas that involve painful schleps. that's schlep blindness.

the phenomenon isn't limited to startups. most people don't consciously decide not to be in as good physical shape as olympic athletes, for example. their unconscious mind decides for them, shrinking from the work involved.

the most striking example i know of schlep blindness is stripe, or rather stripe's idea. for over a decade, every hacker who'd ever had to process payments online knew how painful the experience was. thousands of people must have known about this problem. and yet when they started startups, they decided to build recipe sites, or aggregators for local events. why? why work on problems few care much about and no one will pay for, when you could fix one of the most important components of the world's infrastructure? because schlep blindness prevented people from even considering the idea of fixing payments.

probably no one who applied to y combinator to work on a recipe site began by asking "should we fix payments, or build a recipe site?" and chose the recipe site. though the idea of fixing payments was right there in plain sight, they never saw it, because their unconscious mind shrank from the complications involved. you'd have to make deals with banks. how do you do that? plus you're moving money, so you're going to have to deal with fraud, and people trying to break into your servers. plus there are probably all sorts of regulations to comply with. it's a lot more intimidating to start a startup like this than a recipe site.

that scariness makes ambitious ideas doubly valuable. in addition to their intrinsic value, they're like undervalued stocks in the sense that there's less demand for them among founders. if you pick an ambitious idea, you'll have less competition, because everyone else will have been frightened off by the challenges involved. (this is also true of starting a startup generally.)

how do you overcome schlep blindness? frankly, the most valuable antidote to schlep blindness is probably ignorance. most successful founders would

probably say that if they'd known when they were starting their company about the obstacles they'd have to overcome, they might never have started it. maybe that's one reason the most successful startups of all so often have young founders.

in practice the founders grow with the problems. but no one seems able to foresee that, not even older, more experienced founders. so the reason younger founders have an advantage is that they make two mistakes that cancel each other out. they don't know how much they can grow, but they also don't know how much they'll need to. older founders only make the first mistake.

ignorance can't solve everything though. some ideas so obviously entail alarming schleps that anyone can see them. how do you see ideas like that? the trick i recommend is to take yourself out of the picture. instead of asking "what problem should i solve?" ask "what problem do i wish someone else would solve for me?" if someone who had to process payments before stripe had tried asking that, stripe would have been one of the first things they wished for.

it's too late now to be stripe, but there's plenty still broken in the world, if you know how to see it.

thanks to sam altman, paul buchheit, patrick collison, aaron iba, jessica livingston, emmett shear, and harj taggar for reading drafts of this.

04

frighteningly ambitious startup ideas

paulgraham.com/ambitious.html

march 2012

one of the more surprising things i've noticed while working on y combinator is how frightening the most ambitious startup ideas are. in this essay i'm going to demonstrate this phenomenon by describing some. any one of them could make you a billionaire. that might sound like an attractive prospect, and yet when i describe these ideas you may notice you find yourself shrinking away from them.

don't worry, it's not a sign of weakness. arguably it's a sign of sanity. the biggest startup ideas are terrifying. and not just because they'd be a lot of work. the biggest ideas seem to threaten your identity: you wonder if you'd have enough ambition to carry them through.

there's a scene in *being john malkovich* where the nerdy hero encounters a very attractive, sophisticated woman. she says to him:

> here's the thing: if you ever got me, you wouldn't have a clue what to do with me.

that's what these ideas say to us.

this phenomenon is one of the most important things you can understand about startups. [1] you'd expect big startup ideas to be attractive, but actually they tend to repel you. and that has a bunch of consequences. it means

these ideas are invisible to most people who try to think of startup ideas, because their subconscious filters them out. even the most ambitious people are probably best off approaching them obliquely.

1. a new search engine

the best ideas are just on the right side of impossible. i don't know if this one is possible, but there are signs it might be. making a new search engine means competing with google, and recently i've noticed some cracks in their fortress.

the point when it became clear to me that microsoft had lost their way was when they decided to get into the search business. that was not a natural move for microsoft. they did it because they were afraid of google, and google was in the search business. but this meant (a) google was now setting microsoft's agenda, and (b) microsoft's agenda consisted of stuff they weren't good at.

microsoft : google :: google : facebook.

that does not by itself mean there's room for a new search engine, but lately when using google search i've found myself nostalgic for the old days, when google was true to its own slightly aspy self. google used to give me a page of the right answers, fast, with no clutter. now the results seem inspired by the scientologist principle that what's true is what's true for you. and the pages don't have the clean, sparse feel they used to. google search results used to look like the output of a unix utility. now if i accidentally put the cursor in the wrong place, anything might happen.

the way to win here is to build the search engine all the hackers use. a search engine whose users consisted of the top 10,000 hackers and no one else would be in a very powerful position despite its small size, just as google was when it was that search engine. and for the first time in over a decade the idea of switching seems thinkable to me.

since anyone capable of starting this company is one of those 10,000 hackers, the route is at least straightforward: make the search engine you yourself want. feel free to make it excessively hackerish. make it really good for

code search, for example. would you like search queries to be turing complete? anything that gets you those 10,000 users is ipso facto good.
don't worry if something you want to do will constrain you in the long term, because if you don't get that initial core of users, there won't be a long term. if you can just build something that you and your friends genuinely prefer to google, you're already about 10% of the way to an ipo, just as facebook was (though they probably didn't realize it) when they got all the harvard undergrads.

2. replace email

email was not designed to be used the way we use it now. email is not a messaging protocol. it's a todo list. or rather, my inbox is a todo list, and email is the way things get onto it. but it is a disastrously bad todo list.
i'm open to different types of solutions to this problem, but i suspect that tweaking the inbox is not enough, and that email has to be replaced with a new protocol. this new protocol should be a todo list protocol, not a messaging protocol, although there is a degenerate case where what someone wants you to do is: read the following text.
as a todo list protocol, the new protocol should give more power to the recipient than email does. i want there to be more restrictions on what someone can put on my todo list. and when someone can put something on my todo list, i want them to tell me more about what they want from me. do they want me to do something beyond just reading some text? how important is it? (there obviously has to be some mechanism to prevent people from saying everything is important.) when does it have to be done?
this is one of those ideas that's like an irresistible force meeting an immovable object. on one hand, entrenched protocols are impossible to replace. on the other, it seems unlikely that people in 100 years will still be living in the same email hell we do now. and if email is going to get replaced eventually, why not now?
if you do it right, you may be able to avoid the usual chicken and egg problem new protocols face, because some of the most powerful people in the

world will be among the first to switch to it. they're all at the mercy of email too.

whatever you build, make it fast. gmail has become painfully slow. [2] if you made something no better than gmail, but fast, that alone would let you start to pull users away from gmail.

gmail is slow because google can't afford to spend a lot on it. but people will pay for this. i'd have no problem paying $50 a month. considering how much time i spend in email, it's kind of scary to think how much i'd be justified in paying. at least $1000 a month. if i spend several hours a day reading and writing email, that would be a cheap way to make my life better.

3. replace universities

people are all over this idea lately, and i think they're onto something. i'm reluctant to suggest that an institution that's been around for a millennium is finished just because of some mistakes they made in the last few decades, but certainly in the last few decades us universities seem to have been headed down the wrong path. one could do a lot better for a lot less money.

i don't think universities will disappear. they won't be replaced wholesale. they'll just lose the de facto monopoly on certain types of learning that they once had. there will be many different ways to learn different things, and some may look quite different from universities. y combinator itself is arguably one of them.

learning is such a big problem that changing the way people do it will have a wave of secondary effects. for example, the name of the university one went to is treated by a lot of people (correctly or not) as a credential in its own right. if learning breaks up into many little pieces, credentialling may separate from it. there may even need to be replacements for campus social life (and oddly enough, yc even has aspects of that).

you could replace high schools too, but there you face bureaucratic obstacles that would slow down a startup. universities seem the place to start.

4. internet drama

hollywood has been slow to embrace the internet. that was a mistake, because i think we can now call a winner in the race between delivery mechanisms, and it is the internet, not cable.

a lot of the reason is the horribleness of cable clients, also known as tvs. our family didn't wait for apple tv. we hated our last tv so much that a few months ago we replaced it with an imac bolted to the wall. it's a little inconvenient to control it with a wireless mouse, but the overall experience is much better than the nightmare ui we had to deal with before.

some of the attention people currently devote to watching movies and tv can be stolen by things that seem completely unrelated, like social networking apps. more can be stolen by things that are a little more closely related, like games. but there will probably always remain some residual demand for conventional drama, where you sit passively and watch as a plot happens. so how do you deliver drama via the internet? whatever you make will have to be on a larger scale than youtube clips. when people sit down to watch a show, they want to know what they're going to get: either part of a series with familiar characters, or a single longer "movie" whose basic premise they know in advance.

there are two ways delivery and payment could play out. either some company like netflix or apple will be the app store for entertainment, and you'll reach audiences through them. or the would-be app stores will be too overreaching, or too technically inflexible, and companies will arise to supply payment and streaming a la carte to the producers of drama. if that's the way things play out, there will also be a need for such infrastructure companies.

5. the next steve jobs

i was talking recently to someone who knew apple well, and i asked him if the people now running the company would be able to keep creating new things the way apple had under steve jobs. his answer was simply "no." i already feared that would be the answer. i asked more to see how he'd qualify it. but he didn't qualify it at all. no, there will be no more great new stuff beyond whatever's currently in the pipeline. apple's revenues may continue

to rise for a long time, but as microsoft shows, revenue is a lagging indicator in the technology business.

so if apple's not going to make the next ipad, who is? none of the existing players. none of them are run by product visionaries, and empirically you can't seem to get those by hiring them. empirically the way you get a product visionary as ceo is for him to found the company and not get fired. so the company that creates the next wave of hardware is probably going to have to be a startup.

i realize it sounds preposterously ambitious for a startup to try to become as big as apple. but no more ambitious than it was for apple to become as big as apple, and they did it. plus a startup taking on this problem now has an advantage the original apple didn't: the example of apple. steve jobs has shown us what's possible. that helps would-be successors both directly, as roger bannister did, by showing how much better you can do than people did before, and indirectly, as augustus did, by lodging the idea in users' minds that a single person could unroll the future for them. [3]

now steve is gone there's a vacuum we can all feel. if a new company led boldly into the future of hardware, users would follow. the ceo of that company, the "next steve jobs," might not measure up to steve jobs. but he wouldn't have to. he'd just have to do a better job than samsung and hp and nokia, and that seems pretty doable.

6. bring back moore's law

the last 10 years have reminded us what moore's law actually says. till about 2002 you could safely misinterpret it as promising that clock speeds would double every 18 months. actually what it says is that circuit densities will double every 18 months. it used to seem pedantic to point that out. not any more. intel can no longer give us faster cpus, just more of them.

this moore's law is not as good as the old one. moore's law used to mean that if your software was slow, all you had to do was wait, and the inexorable progress of hardware would solve your problems. now if your softw-

are is slow you have to rewrite it to do more things in parallel, which is a lot more work than waiting.

it would be great if a startup could give us something of the old moore's law back, by writing software that could make a large number of cpus look to the developer like one very fast cpu. there are several ways to approach this problem. the most ambitious is to try to do it automatically: to write a compiler that will parallelize our code for us. there's a name for this compiler, *the sufficiently smart compiler,* and it is a byword for impossibility. but is it really impossible? is there no configuration of the bits in memory of a present day computer that is this compiler? if you really think so, you should try to prove it, because that would be an interesting result. and if it's not impossible but simply very hard, it might be worth trying to write it. the expected value would be high even if the chance of succeeding was low.

the reason the expected value is so high is web services. if you could write software that gave programmers the convenience of the way things were in the old days, you could offer it to them as a web service. and that would in turn mean that you got practically all the users.

imagine there was another processor manufacturer that could still translate increased circuit densities into increased clock speeds. they'd take most of intel's business. and since web services mean that no one sees their processors anymore, by writing the sufficiently smart compiler you could create a situation indistinguishable from you being that manufacturer, at least for the server market.

the least ambitious way of approaching the problem is to start from the other end, and offer programmers more parallelizable lego blocks to build programs out of, like hadoop and mapreduce. then the programmer still does much of the work of optimization.

there's an intriguing middle ground where you build a semi-automatic weapon—where there's a human in the loop. you make something that looks to the user like the sufficiently smart compiler, but inside has people, using highly developed optimization tools to find and eliminate bottlenecks

in users' programs. these people might be your employees, or you might create a marketplace for optimization.

an optimization marketplace would be a way to generate the sufficiently smart compiler piecemeal, because participants would immediately start writing bots. it would be a curious state of affairs if you could get to the point where everything could be done by bots, because then you'd have made the sufficiently smart compiler, but no one person would have a complete copy of it.

i realize how crazy all this sounds. in fact, what i like about this idea is all the different ways in which it's wrong. the whole idea of focusing on optimization is counter to the general trend in software development for the last several decades. trying to write the sufficiently smart compiler is by definition a mistake. and even if it weren't, compilers are the sort of software that's supposed to be created by open source projects, not companies. plus if this works it will deprive all the programmers who take pleasure in making multithreaded apps of so much amusing complexity. the forum troll i have by now internalized doesn't even know where to begin in raising objections to this project. now that's what i call a startup idea.

7. ongoing diagnosis

but wait, here's another that could face even greater resistance: ongoing, automatic medical diagnosis.

one of my tricks for generating startup ideas is to imagine the ways in which we'll seem backward to future generations. and i'm pretty sure that to people 50 or 100 years in the future, it will seem barbaric that people in our era waited till they had symptoms to be diagnosed with conditions like heart disease and cancer.

for example, in 2004 bill clinton found he was feeling short of breath. doctors discovered that several of his arteries were over 90% blocked and 3 days later he had a quadruple bypass. it seems reasonable to assume bill clinton has the best medical care available. and yet even he had to wait till his arteries were over 90% blocked to learn that the number was over 90%. surely at some point in the future we'll know these numbers the way

we now know something like our weight. ditto for cancer. it will seem preposterous to future generations that we wait till patients have physical symptoms to be diagnosed with cancer. cancer will show up on some sort of radar screen immediately.

(of course, what shows up on the radar screen may be different from what we think of now as cancer. i wouldn't be surprised if at any given time we have ten or even hundreds of microcancers going at once, none of which normally amount to anything.)

a lot of the obstacles to ongoing diagnosis will come from the fact that it's going against the grain of the medical profession. the way medicine has always worked is that patients come to doctors with problems, and the doctors figure out what's wrong. a lot of doctors don't like the idea of going on the medical equivalent of what lawyers call a "fishing expedition," where you go looking for problems without knowing what you're looking for. they call the things that get discovered this way "incidentalomas," and they are something of a nuisance.

for example, a friend of mine once had her brain scanned as part of a study. she was horrified when the doctors running the study discovered what appeared to be a large tumor. after further testing, it turned out to be a harmless cyst. but it cost her a few days of terror. a lot of doctors worry that if you start scanning people with no symptoms, you'll get this on a giant scale: a huge number of false alarms that make patients panic and require expensive and perhaps even dangerous tests to resolve. but i think that's just an artifact of current limitations. if people were scanned all the time and we got better at deciding what was a real problem, my friend would have known about this cyst her whole life and known it was harmless, just as we do a birthmark.

there is room for a lot of startups here. in addition to the technical obstacles all startups face, and the bureaucratic obstacles all medical startups face, they'll be going against thousands of years of medical tradition. but it will happen, and it will be a great thing—so great that people in the future

will feel as sorry for us as we do for the generations that lived before anaesthesia and antibiotics.

tactics

let me conclude with some tactical advice. if you want to take on a problem as big as the ones i've discussed, don't make a direct frontal attack on it. don't say, for example, that you're going to replace email. if you do that you raise too many expectations. your employees and investors will constantly be asking "are we there yet?" and you'll have an army of haters waiting to see you fail. just say you're building todo-list software. that sounds harmless. people can notice you've replaced email when it's a *fait accompli*. [4]

empirically, the way to do really big things seems to be to start with deceptively small things. want to dominate microcomputer software? start by writing a basic interpreter for a machine with a few thousand users. want to make the universal web site? start by building a site for harvard undergrads to stalk one another.

empirically, it's not just for other people that you need to start small. you need to for your own sake. neither bill gates nor mark zuckerberg knew at first how big their companies were going to get. all they knew was that they were onto something. maybe it's a bad idea to have really big ambitions initially, because the bigger your ambition, the longer it's going to take, and the further you project into the future, the more likely you'll get it wrong.

i think the way to use these big ideas is not to try to identify a precise point in the future and then ask yourself how to get from here to there, like the popular image of a visionary. you'll be better off if you operate like columbus and just head in a general westerly direction. don't try to construct the future like a building, because your current blueprint is almost certainly mistaken. start with something you know works, and when you expand, expand westward.

the popular image of the visionary is someone with a clear view of the future, but empirically it may be better to have a blurry one.

notes

1 it's also one of the most important things vcs fail to understand about startups. most expect founders to walk in with a clear plan for the future, and judge them based on that. few consciously realize that in the biggest successes there is the least correlation between the initial plan and what the startup eventually becomes.

2 this sentence originally read "gmail is painfully slow." thanks to paul buchheit for the correction.

3 roger bannister is famous as the first person to run a mile in under 4 minutes. but his world record only lasted 46 days. once he showed it could be done, lots of others followed. ten years later jim ryun ran a 3:59 mile as a high school junior.

4 if you want to be the next apple, maybe you don't even want to start with consumer electronics. maybe at first you make something hackers use. or you make something popular but apparently unimportant, like a headset or router. all you need is a bridgehead.

thanks to sam altman, trevor blackwell, paul buchheit, patrick collison, aaron iba, jessica livingston, robert morris, harj taggar and garry tan for reading drafts of this.

05

do things that don't scale

paulgraham.com/ds.html

july 2013

one of the most common types of advice we give at y combinator is to do things that don't scale. a lot of would-be founders believe that startups either take off or don't. you build something, make it available, and if you've made a better mousetrap, people beat a path to your door as promised. or they don't, in which case the market must not exist. [1]

actually startups take off because the founders make them take off. there may be a handful that just grew by themselves, but usually it takes some sort of push to get them going. a good metaphor would be the cranks that car engines had before they got electric starters. once the engine was going, it would keep going, but there was a separate and laborious process to get it going.

recruit

the most common unscalable thing founders have to do at the start is to recruit users manually. nearly all startups have to. you can't wait for users to come to you. you have to go out and get them.

stripe is one of the most successful startups we've funded, and the problem they solved was an urgent one. if anyone could have sat back and waited for users, it was stripe. but in fact they're famous within yc for aggressive early user acquisition.

startups building things for other startups have a big pool of potential users in the other companies we've funded, and none took better advantage of it

than stripe. at yc we use the term "collison installation" for the technique they invented. more diffident founders ask "will you try our beta?" and if the answer is yes, they say "great, we'll send you a link." but the collison brothers weren't going to wait. when anyone agreed to try stripe they'd say "right then, give me your laptop" and set them up on the spot.

there are two reasons founders resist going out and recruiting users individually. one is a combination of shyness and laziness. they'd rather sit at home writing code than go out and talk to a bunch of strangers and probably be rejected by most of them. but for a startup to succeed, at least one founder (usually the ceo) will have to spend a lot of time on sales and marketing. [2]

the other reason founders ignore this path is that the absolute numbers seem so small at first. this can't be how the big, famous startups got started, they think. the mistake they make is to underestimate the power of compound growth. we encourage every startup to measure their progress by weekly growth rate. if you have 100 users, you need to get 10 more next week to grow 10% a week. and while 110 may not seem much better than 100, if you keep growing at 10% a week you'll be surprised how big the numbers get. after a year you'll have 14,000 users, and after 2 years you'll have 2 million.

you'll be doing different things when you're acquiring users a thousand at a time, and growth has to slow down eventually. but if the market exists you can usually start by recruiting users manually and then gradually switch to less manual methods. [3]

airbnb is a classic example of this technique. marketplaces are so hard to get rolling that you should expect to take heroic measures at first. in airbnb's case, these consisted of going door to door in new york, recruiting new users and helping existing ones improve their listings. when i remember the airbnbs during yc, i picture them with rolly bags, because when they showed up for tuesday dinners they'd always just flown back from somewhere.

fragile

airbnb now seems like an unstoppable juggernaut, but early on it was so fragile that about 30 days of going out and engaging in person with users made the difference between success and failure.

that initial fragility was not a unique feature of airbnb. almost all startups are fragile initially. and that's one of the biggest things inexperienced founders and investors (and reporters and know-it-alls on forums) get wrong about them. they unconsciously judge larval startups by the standards of established ones. they're like someone looking at a newborn baby and concluding "there's no way this tiny creature could ever accomplish anything."

it's harmless if reporters and know-it-alls dismiss your startup. they always get things wrong. it's even ok if investors dismiss your startup; they'll change their minds when they see growth. the big danger is that you'll dismiss your startup yourself. i've seen it happen. i often have to encourage founders who don't see the full potential of what they're building. even bill gates made that mistake. he returned to harvard for the fall semester after starting microsoft. he didn't stay long, but he wouldn't have returned at all if he'd realized microsoft was going to be even a fraction of the size it turned out to be. [4]

the question to ask about an early stage startup is not "is this company taking over the world?" but "how big could this company get if the founders did the right things?" and the right things often seem both laborious and inconsequential at the time. microsoft can't have seemed very impressive when it was just a couple guys in albuquerque writing basic interpreters for a market of a few thousand hobbyists (as they were then called), but in retrospect that was the optimal path to dominating microcomputer software. and i know brian chesky and joe gebbia didn't feel like they were en route to the big time as they were taking "professional" photos of their first hosts' apartments. they were just trying to survive. but in retrospect that too was the optimal path to dominating a big market.

how do you find users to recruit manually? if you build something to solve your own problems, then you only have to find your peers, which is usually

straightforward. otherwise you'll have to make a more deliberate effort to locate the most promising vein of users. the usual way to do that is to get some initial set of users by doing a comparatively untargeted launch, and then to observe which kind seem most enthusiastic, and seek out more like them. for example, ben silbermann noticed that a lot of the earliest pinterest users were interested in design, so he went to a conference of design bloggers to recruit users, and that worked well. [5]

delight

you should take extraordinary measures not just to acquire users, but also to make them happy. for as long as they could (which turned out to be surprisingly long), wufoo sent each new user a hand-written thank you note. your first users should feel that signing up with you was one of the best choices they ever made. and you in turn should be racking your brains to think of new ways to delight them.

why do we have to teach startups this? why is it counterintuitive for founders? three reasons, i think.

one is that a lot of startup founders are trained as engineers, and customer service is not part of the training of engineers. you're supposed to build things that are robust and elegant, not be slavishly attentive to individual users like some kind of salesperson. ironically, part of the reason engineering is traditionally averse to handholding is that its traditions date from a time when engineers were less powerful — when they were only in charge of their narrow domain of building things, rather than running the whole show. you can be ornery when you're scotty, but not when you're kirk.

another reason founders don't focus enough on individual customers is that they worry it won't scale. but when founders of larval startups worry about this, i point out that in their current state they have nothing to lose. maybe if they go out of their way to make existing users super happy, they'll one day have too many to do so much for. that would be a great problem to have. see if you can make it happen. and incidentally, when it does, you'll find that delighting customers scales better than you expected. partly because you can usually find ways to make anything scale more than you

would have predicted, and partly because delighting customers will by then have permeated your culture.

i have never once seen a startup lured down a blind alley by trying too hard to make their initial users happy.

but perhaps the biggest thing preventing founders from realizing how attentive they could be to their users is that they've never experienced such attention themselves. their standards for customer service have been set by the companies they've been customers of, which are mostly big ones. tim cook doesn't send you a hand-written note after you buy a laptop. he can't. but you can. that's one advantage of being small: you can provide a level of service no big company can. [6]

once you realize that existing conventions are not the upper bound on user experience, it's interesting in a very pleasant way to think about how far you could go to delight your users.

experience

i was trying to think of a phrase to convey how extreme your attention to users should be, and i realized steve jobs had already done it: insanely great. steve wasn't just using "insanely" as a synonym for "very." he meant it more literally — that one should focus on quality of execution to a degree that in everyday life would be considered pathological.

all the most successful startups we've funded have, and that probably doesn't surprise would-be founders. what novice founders don't get is what insanely great translates to in a larval startup. when steve jobs started using that phrase, apple was already an established company. he meant the mac (and its documentation and even packaging — such is the nature of obsession) should be insanely well designed and manufactured. that's not hard for engineers to grasp. it's just a more extreme version of designing a robust and elegant product.

what founders have a hard time grasping (and steve himself might have had a hard time grasping) is what insanely great morphs into as you roll the time slider back to the first couple months of a startup's life. it's not the product that should be insanely great, but the experience of being your

user. the product is just one component of that. for a big company it's necessarily the dominant one. but you can and should give users an insanely great experience with an early, incomplete, buggy product, if you make up the difference with attentiveness.

can, perhaps, but should? yes. over-engaging with early users is not just a permissible technique for getting growth rolling. for most successful startups it's a necessary part of the feedback loop that makes the product good. making a better mousetrap is not an atomic operation. even if you start the way most successful startups have, by building something you yourself need, the first thing you build is never quite right. and except in domains with big penalties for making mistakes, it's often better not to aim for perfection initially. in software, especially, it usually works best to get something in front of users as soon as it has a quantum of utility, and then see what they do with it. perfectionism is often an excuse for procrastination, and in any case your initial model of users is always inaccurate, even if you're one of them. [7]

the feedback you get from engaging directly with your earliest users will be the best you ever get. when you're so big you have to resort to focus groups, you'll wish you could go over to your users' homes and offices and watch them use your stuff like you did when there were only a handful of them.

fire

sometimes the right unscalable trick is to focus on a deliberately narrow market. it's like keeping a fire contained at first to get it really hot before adding more logs.

that's what facebook did. at first it was just for harvard students. in that form it only had a potential market of a few thousand people, but because they felt it was really for them, a critical mass of them signed up. after facebook stopped being for harvard students, it remained for students at specific colleges for quite a while. when i interviewed mark zuckerberg at startup school, he said that while it was a lot of work creating course lists

for each school, doing that made students feel the site was their natural home.

any startup that could be described as a marketplace usually has to start in a subset of the market, but this can work for other startups as well. it's always worth asking if there's a subset of the market in which you can get a critical mass of users quickly. [8]

most startups that use the contained fire strategy do it unconsciously. they build something for themselves and their friends, who happen to be the early adopters, and only realize later that they could offer it to a broader market. the strategy works just as well if you do it unconsciously. the biggest danger of not being consciously aware of this pattern is for those who naively discard part of it. e.g. if you don't build something for yourself and your friends, or even if you do, but you come from the corporate world and your friends are not early adopters, you'll no longer have a perfect initial market handed to you on a platter.

among companies, the best early adopters are usually other startups. they're more open to new things both by nature and because, having just been started, they haven't made all their choices yet. plus when they succeed they grow fast, and you with them. it was one of many unforeseen advantages of the yc model (and specifically of making yc big) that b2b startups now have an instant market of hundreds of other startups ready at hand.

meraki

for hardware startups there's a variant of doing things that don't scale that we call "pulling a meraki." although we didn't fund meraki, the founders were robert morris's grad students, so we know their history. they got started by doing something that really doesn't scale: assembling their routers themselves.

hardware startups face an obstacle that software startups don't. the minimum order for a factory production run is usually several hundred thousand dollars. which can put you in a catch-22: without a product you can't generate the growth you need to raise the money to manufacture your

product. back when hardware startups had to rely on investors for money, you had to be pretty convincing to overcome this. the arrival of crowdfunding (or more precisely, preorders) has helped a lot. but even so i'd advise startups to pull a meraki initially if they can. that's what pebble did. the pebbles assembled the first several hundred watches themselves. if they hadn't gone through that phase, they probably wouldn't have sold $10 million worth of watches when they did go on kickstarter.

like paying excessive attention to early customers, fabricating things yourself turns out to be valuable for hardware startups. you can tweak the design faster when you're the factory, and you learn things you'd never have known otherwise. eric migicovsky of pebble said one of the things he learned was "how valuable it was to source good screws." who knew?

consult

sometimes we advise founders of b2b startups to take over-engagement to an extreme, and to pick a single user and act as if they were consultants building something just for that one user. the initial user serves as the form for your mold; keep tweaking till you fit their needs perfectly, and you'll usually find you've made something other users want too. even if there aren't many of them, there are probably adjacent territories that have more. as long as you can find just one user who really needs something and can act on that need, you've got a toehold in making something people want, and that's as much as any startup needs initially. [9]

consulting is the canonical example of work that doesn't scale. but (like other ways of bestowing one's favors liberally) it's safe to do it so long as you're not being paid to. that's where companies cross the line. so long as you're a product company that's merely being extra attentive to a customer, they're very grateful even if you don't solve all their problems. but when they start paying you specifically for that attentiveness — when they start paying you by the hour — they expect you to do everything.

another consulting-like technique for recruiting initially lukewarm users is to use your software yourselves on their behalf. we did that at viaweb. when we approached merchants asking if they wanted to use our software

to make online stores, some said no, but they'd let us make one for them. since we would do anything to get users, we did. we felt pretty lame at the time. instead of organizing big strategic e-commerce partnerships, we were trying to sell luggage and pens and men's shirts. but in retrospect it was exactly the right thing to do, because it taught us how it would feel to merchants to use our software. sometimes the feedback loop was near instantaneous: in the middle of building some merchant's site i'd find i needed a feature we didn't have, so i'd spend a couple hours implementing it and then resume building the site.

manual

there's a more extreme variant where you don't just use your software, but are your software. when you only have a small number of users, you can sometimes get away with doing by hand things that you plan to automate later. this lets you launch faster, and when you do finally automate yourself out of the loop, you'll know exactly what to build because you'll have muscle memory from doing it yourself.

when manual components look to the user like software, this technique starts to have aspects of a practical joke. for example, the way stripe delivered "instant" merchant accounts to its first users was that the founders manually signed them up for traditional merchant accounts behind the scenes.

some startups could be entirely manual at first. if you can find someone with a problem that needs solving and you can solve it manually, go ahead and do that for as long as you can, and then gradually automate the bottlenecks. it would be a little frightening to be solving users' problems in a way that wasn't yet automatic, but less frightening than the far more common case of having something automatic that doesn't yet solve anyone's problems.

big

i should mention one sort of initial tactic that usually doesn't work: the big launch. i occasionally meet founders who seem to believe startups are projectiles rather than powered aircraft, and that they'll make it big if and only

if they're launched with sufficient initial velocity. they want to launch simultaneously in 8 different publications, with embargoes. and on a tuesday, of course, since they read somewhere that's the optimum day to launch something.

it's easy to see how little launches matter. think of some successful startups. how many of their launches do you remember? all you need from a launch is some initial core of users. how well you're doing a few months later will depend more on how happy you made those users than how many there were of them. [10]

so why do founders think launches matter? a combination of solipsism and laziness. they think what they're building is so great that everyone who hears about it will immediately sign up. plus it would be so much less work if you could get users merely by broadcasting your existence, rather than recruiting them one at a time. but even if what you're building really is great, getting users will always be a gradual process — partly because great things are usually also novel, but mainly because users have other things to think about.

partnerships too usually don't work. they don't work for startups in general, but they especially don't work as a way to get growth started. it's a common mistake among inexperienced founders to believe that a partnership with a big company will be their big break. six months later they're all saying the same thing: that was way more work than we expected, and we ended up getting practically nothing out of it. [11]

it's not enough just to do something extraordinary initially. you have to make an extraordinary *effort* initially. any strategy that omits the effort — whether it's expecting a big launch to get you users, or a big partner — is ipso facto suspect.

vector

the need to do something unscalably laborious to get started is so nearly universal that it might be a good idea to stop thinking of startup ideas as scalars. instead we should try thinking of them as pairs of what you're goi-

ng to build, plus the unscalable thing(s) you're going to do initially to get the company going.

it could be interesting to start viewing startup ideas this way, because now that there are two components you can try to be imaginative about the second as well as the first. but in most cases the second component will be what it usually is — recruit users manually and give them an overwhelmingly good experience — and the main benefit of treating startups as vectors will be to remind founders they need to work hard in two dimensions. [12]

in the best case, both components of the vector contribute to your company's dna: the unscalable things you have to do to get started are not merely a necessary evil, but change the company permanently for the better. if you have to be aggressive about user acquisition when you're small, you'll probably still be aggressive when you're big. if you have to manufacture your own hardware, or use your software on users's behalf, you'll learn things you couldn't have learned otherwise. and most importantly, if you have to work hard to delight users when you only have a handful of them, you'll keep doing it when you have a lot.

---

notes

1 actually emerson never mentioned mousetraps specifically. he wrote "if a man has good corn or wood, or boards, or pigs, to sell, or can make better chairs or knives, crucibles or church organs, than anybody else, you will find a broad hard-beaten road to his house, though it be in the woods."

2 thanks to sam altman for suggesting i make this explicit. and no, you can't avoid doing sales by hiring someone to do it for you. you have to do sales yourself initially. later you can hire a real salesperson to replace you.

3 the reason this works is that as you get bigger, your size helps you grow. patrick collison wrote "at some point, there was a very noticeable change in how stripe felt. it tipped from being this boulder we had to push to being a train car that in fact had its own momentum."

4 one of the more subtle ways in which yc can help founders is by calibrating their ambitions, because we know exactly how a lot of successful startups looked when they were just getting started.

5 if you're building something for which you can't easily get a small set of users to observe — e.g. enterprise software — and in a domain where you have no connections, you'll have to rely on cold calls and introductions. but should you even be working on such an idea?

6 garry tan pointed out an interesting trap founders fall into in the beginning. they want so much to seem big that they imitate even the flaws of big companies, like indifference to individual users. this seems to them more "professional." actually it's better to embrace the fact that you're small and use whatever advantages that brings.

7 your user model almost couldn't be perfectly accurate, because users' needs often change in response to what you build for them. build them a microcomputer, and suddenly they need to run spreadsheets on it, because the arrival of your new microcomputer causes someone to invent the spreadsheet.

8 if you have to choose between the subset that will sign up quickest and those that will pay the most, it's usually best to pick the former, because those are probably the early adopters. they'll have a better influence on your product, and they won't make you expend as much effort on sales. and though they have less money, you don't need that much to maintain your target growth rate early on.

9 yes, i can imagine cases where you could end up making something that was really only useful for one user. but those are usually obvious, even to inexperienced founders. so if it's not obvious you'd be making something for a market of one, don't worry about that danger.

10 there may even be an inverse correlation between launch magnitude and success. the only launches i remember are famous flops like the segway and google wave. wave is a particularly alarming example, because i think it was actually a great idea that was killed partly by its overdone launch.

11 google grew big on the back of yahoo, but that wasn't a partnership. yahoo was their customer.

12 it will also remind founders that an idea where the second component is empty — an idea where there is nothing you can do to get going, e.g. because you have no way to find users to recruit manually — is probably a bad idea, at least for those founders.

thanks to sam altman, paul buchheit, patrick collison, kevin hale, steven levy, jessica livingston, geoff ralston, and garry tan for reading drafts of this.

06

startup = growth

paulgraham.com/growth.html

september 2012

a startup is a company designed to grow fast. being newly founded does not in itself make a company a startup. nor is it necessary for a startup to work on technology, or take venture funding, or have some sort of "exit." the only essential thing is growth. everything else we associate with startups follows from growth.

if you want to start one it's important to understand that. startups are so hard that you can't be pointed off to the side and hope to succeed. you have to know that growth is what you're after. the good news is, if you get growth, everything else tends to fall into place. which means you can use growth like a compass to make almost every decision you face.

redwoods

let's start with a distinction that should be obvious but is often overlooked: not every newly founded company is a startup. millions of companies are started every year in the us. only a tiny fraction are startups. most are service businesses — restaurants, barbershops, plumbers, and so on. these are not startups, except in a few unusual cases. a barbershop isn't designed to grow fast. whereas a search engine, for example, is.

when i say startups are designed to grow fast, i mean it in two senses. partly i mean designed in the sense of intended, because most startups fail. but i also mean startups are different by nature, in the same way a redwood seedling has a different destiny from a bean sprout.

that difference is why there's a distinct word, "startup," for companies designed to grow fast. if all companies were essentially similar, but some through luck or the efforts of their founders ended up growing very fast, we wouldn't need a separate word. we could just talk about super-successful companies and less successful ones. but in fact startups do have a different sort of dna from other businesses. google is not just a barbershop whose founders were unusually lucky and hard-working. google was different from the beginning.

to grow rapidly, you need to make something you can sell to a big market. that's the difference between google and a barbershop. a barbershop doesn't scale.

for a company to grow really big, it must (a) make something lots of people want, and (b) reach and serve all those people. barbershops are doing fine in the (a) department. almost everyone needs their hair cut. the problem for a barbershop, as for any retail establishment, is (b). a barbershop serves customers in person, and few will travel far for a haircut. and even if they did, the barbershop couldn't accommodate them. [1]

writing software is a great way to solve (b), but you can still end up constrained in (a). if you write software to teach tibetan to hungarian speakers, you'll be able to reach most of the people who want it, but there won't be many of them. if you make software to teach english to chinese speakers, however, you're in startup territory.

most businesses are tightly constrained in (a) or (b). the distinctive feature of successful startups is that they're not.

ideas

it might seem that it would always be better to start a startup than an ordinary business. if you're going to start a company, why not start the type with the most potential? the catch is that this is a (fairly) efficient market. if you write software to teach tibetan to hungarians, you won't have much competition. if you write software to teach english to chinese speakers, you'll face ferocious competition, precisely because that's such a larger prize. [2]

the constraints that limit ordinary companies also protect them. that's the tradeoff. if you start a barbershop, you only have to compete with other local barbers. if you start a search engine you have to compete with the whole world.

the most important thing that the constraints on a normal business protect it from is not competition, however, but the difficulty of coming up with new ideas. if you open a bar in a particular neighborhood, as well as limiting your potential and protecting you from competitors, that geographic constraint also helps define your company. bar + neighborhood is a sufficient idea for a small business. similarly for companies constrained in (a). your niche both protects and defines you.

whereas if you want to start a startup, you're probably going to have to think of something fairly novel. a startup has to make something it can deliver to a large market, and ideas of that type are so valuable that all the obvious ones are already taken.

that space of ideas has been so thoroughly picked over that a startup generally has to work on something everyone else has overlooked. i was going to write that one has to make a conscious effort to find ideas everyone else has overlooked. but that's not how most startups get started. usually successful startups happen because the founders are sufficiently different from other people that ideas few others can see seem obvious to them. perhaps later they step back and notice they've found an idea in everyone else's blind spot, and from that point make a deliberate effort to stay there. [3] but at the moment when successful startups get started, much of the innovation is unconscious.

what's different about successful founders is that they can see different problems. it's a particularly good combination both to be good at technology and to face problems that can be solved by it, because technology changes so rapidly that formerly bad ideas often become good without anyone noticing. steve wozniak's problem was that he wanted his own computer. that was an unusual problem to have in 1975. but technological change was about to make it a much more common one. because he not

only wanted a computer but knew how to build them, wozniak was able to make himself one. and the problem he solved for himself became one that apple solved for millions of people in the coming years. but by the time it was obvious to ordinary people that this was a big market, apple was already established.

google has similar origins. larry page and sergey brin wanted to search the web. but unlike most people they had the technical expertise both to notice that existing search engines were not as good as they could be, and to know how to improve them. over the next few years their problem became everyone's problem, as the web grew to a size where you didn't have to be a picky search expert to notice the old algorithms weren't good enough. but as happened with apple, by the time everyone else realized how important search was, google was entrenched.

that's one connection between startup ideas and technology. rapid change in one area uncovers big, soluble problems in other areas. sometimes the changes are advances, and what they change is solubility. that was the kind of change that yielded apple; advances in chip technology finally let steve wozniak design a computer he could afford. but in google's case the most important change was the growth of the web. what changed there was not solubility but bigness.

the other connection between startups and technology is that startups create new ways of doing things, and new ways of doing things are, in the broader sense of the word, new technology. when a startup both begins with an idea exposed by technological change and makes a product consisting of technology in the narrower sense (what used to be called "high technology"), it's easy to conflate the two. but the two connections are distinct and in principle one could start a startup that was neither driven by technological change, nor whose product consisted of technology except in the broader sense. [4]

rate

how fast does a company have to grow to be considered a startup? there's no precise answer to that. "startup" is a pole, not a threshold. starting one

is at first no more than a declaration of one's ambitions. you're committing not just to starting a company, but to starting a fast growing one, and you're thus committing to search for one of the rare ideas of that type. but at first you have no more than commitment. starting a startup is like being an actor in that respect. "actor" too is a pole rather than a threshold. at the beginning of his career, an actor is a waiter who goes to auditions. getting work makes him a successful actor, but he doesn't only become an actor when he's successful.

so the real question is not what growth rate makes a company a startup, but what growth rate successful startups tend to have. for founders that's more than a theoretical question, because it's equivalent to asking if they're on the right path.

the growth of a successful startup usually has three phases:

1. there's an initial period of slow or no growth while the startup tries to figure out what it's doing.
2. as the startup figures out how to make something lots of people want and how to reach those people, there's a period of rapid growth.
3. eventually a successful startup will grow into a big company. growth will slow, partly due to internal limits and partly because the company is starting to bump up against the limits of the markets it serves. [5]

together these three phases produce an s-curve. the phase whose growth defines the startup is the second one, the ascent. its length and slope determine how big the company will be.

the slope is the company's growth rate. if there's one number every founder should always know, it's the company's growth rate. that's the measure of a startup. if you don't know that number, you don't even know if you're doing well or badly.

when i first meet founders and ask what their growth rate is, sometimes they tell me "we get about a hundred new customers a month." that's not a rate. what matters is not the absolute number of new customers, but the ratio of new customers to existing ones. if you're really getting a constant

number of new customers every month, you're in trouble, because that means your growth rate is decreasing.

during y combinator we measure growth rate per week, partly because there is so little time before demo day, and partly because startups early on need frequent feedback from their users to tweak what they're doing. [6]

a good growth rate during yc is 5-7% a week. if you can hit 10% a week you're doing exceptionally well. if you can only manage 1%, it's a sign you haven't yet figured out what you're doing.

the best thing to measure the growth rate of is revenue. the next best, for startups that aren't charging initially, is active users. that's a reasonable proxy for revenue growth because whenever the startup does start trying to make money, their revenues will probably be a constant multiple of active users. [7]

compass

we usually advise startups to pick a growth rate they think they can hit, and then just try to hit it every week. the key word here is "just." if they decide to grow at 7% a week and they hit that number, they're successful for that week. there's nothing more they need to do. but if they don't hit it, they've failed in the only thing that mattered, and should be correspondingly alarmed.

programmers will recognize what we're doing here. we're turning starting a startup into an optimization problem. and anyone who has tried optimizing code knows how wonderfully effective that sort of narrow focus can be. optimizing code means taking an existing program and changing it to use less of something, usually time or memory. you don't have to think about what the program should do, just make it faster. for most programmers this is very satisfying work. the narrow focus makes it a sort of puzzle, and you're generally surprised how fast you can solve it.

focusing on hitting a growth rate reduces the otherwise bewilderingly multifarious problem of starting a startup to a single problem. you can use that target growth rate to make all your decisions for you; anything that gets you the growth you need is ipso facto right. should you spend two days at

a conference? should you hire another programmer? should you focus more on marketing? should you spend time courting some big customer? should you add x feature? whatever gets you your target growth rate. [8]
judging yourself by weekly growth doesn't mean you can look no more than a week ahead. once you experience the pain of missing your target one week (it was the only thing that mattered, and you failed at it), you become interested in anything that could spare you such pain in the future. so you'll be willing for example to hire another programmer, who won't contribute to this week's growth but perhaps in a month will have implemented some new feature that will get you more users. but only if (a) the distraction of hiring someone won't make you miss your numbers in the short term, and (b) you're sufficiently worried about whether you can keep hitting your numbers without hiring someone new.
it's not that you don't think about the future, just that you think about it no more than necessary.
in theory this sort of hill-climbing could get a startup into trouble. they could end up on a local maximum. but in practice that never happens. having to hit a growth number every week forces founders to act, and acting versus not acting is the high bit of succeeding. nine times out of ten, sitting around strategizing is just a form of procrastination. whereas founders' intuitions about which hill to climb are usually better than they realize. plus the maxima in the space of startup ideas are not spiky and isolated. most fairly good ideas are adjacent to even better ones.
the fascinating thing about optimizing for growth is that it can actually discover startup ideas. you can use the need for growth as a form of evolutionary pressure. if you start out with some initial plan and modify it as necessary to keep hitting, say, 10% weekly growth, you may end up with a quite different company than you meant to start. but anything that grows consistently at 10% a week is almost certainly a better idea than you started with.
there's a parallel here to small businesses. just as the constraint of being located in a particular neighborhood helps define a bar, the constraint of

growing at a certain rate can help define a startup.
you'll generally do best to follow that constraint wherever it leads rather than being influenced by some initial vision, just as a scientist is better off following the truth wherever it leads rather than being influenced by what he wishes were the case. when richard feynman said that the imagination of nature was greater than the imagination of man, he meant that if you just keep following the truth you'll discover cooler things than you could ever have made up. for startups, growth is a constraint much like truth. every successful startup is at least partly a product of the imagination of growth. [9]

value

it's hard to find something that grows consistently at several percent a week, but if you do you may have found something surprisingly valuable. if we project forward we see why.

weeklyyearly 1%1.7x 2%2.8x 5%12.6x 7%33.7x 10%142.0x

a company that grows at 1% a week will grow 1.7x a year, whereas a company that grows at 5% a week will grow 12.6x. a company making $1000 a month (a typical number early in yc) and growing at 1% a week will 4 years later be making $7900 a month, which is less than a good programmer makes in salary in silicon valley. a startup that grows at 5% a week will in 4 years be making $25 million a month. [10]
our ancestors must rarely have encountered cases of exponential growth, because our intuitions are no guide here. what happens to fast growing startups tends to surprise even the founders.
small variations in growth rate produce qualitatively different outcomes. that's why there's a separate word for startups, and why startups do things that ordinary companies don't, like raising money and getting acquired. and, strangely enough, it's also why they fail so frequently.
considering how valuable a successful startup can become, anyone familiar with the concept of expected value would be surprised if the failure rate weren't high. if a successful startup could make a founder $100 million, then even if the chance of succeeding were only 1%, the expected value of

starting one would be $1 million. and the probability of a group of sufficiently smart and determined founders succeeding on that scale might be significantly over 1%. for the right people — e.g. the young bill gates — the probability might be 20% or even 50%. so it's not surprising that so many want to take a shot at it. in an efficient market, the number of failed startups should be proportionate to the size of the successes. and since the latter is huge the former should be too. [11]

what this means is that at any given time, the great majority of startups will be working on something that's never going to go anywhere, and yet glorifying their doomed efforts with the grandiose title of "startup."

this doesn't bother me. it's the same with other high-beta vocations, like being an actor or a novelist. i've long since gotten used to it. but it seems to bother a lot of people, particularly those who've started ordinary businesses. many are annoyed that these so-called startups get all the attention, when hardly any of them will amount to anything.

if they stepped back and looked at the whole picture they might be less indignant. the mistake they're making is that by basing their opinions on anecdotal evidence they're implicitly judging by the median rather than the average. if you judge by the median startup, the whole concept of a startup seems like a fraud. you have to invent a bubble to explain why founders want to start them or investors want to fund them. but it's a mistake to use the median in a domain with so much variation. if you look at the average outcome rather than the median, you can understand why investors like them, and why, if they aren't median people, it's a rational choice for founders to start them.

deals

why do investors like startups so much? why are they so hot to invest in photo-sharing apps, rather than solid money-making businesses? not only for the obvious reason.

the test of any investment is the ratio of return to risk. startups pass that test because although they're appallingly risky, the returns when they do succeed are so high. but that's not the only reason investors like startups.

an ordinary slower-growing business might have just as good a ratio of return to risk, if both were lower. so why are vcs interested only in high-growth companies? the reason is that they get paid by getting their capital back, ideally after the startup ipos, or failing that when it's acquired.
the other way to get returns from an investment is in the form of dividends. why isn't there a parallel vc industry that invests in ordinary companies in return for a percentage of their profits? because it's too easy for people who control a private company to funnel its revenues to themselves (e.g. by buying overpriced components from a supplier they control) while making it look like the company is making little profit. anyone who invested in private companies in return for dividends would have to pay close attention to their books.
the reason vcs like to invest in startups is not simply the returns, but also because such investments are so easy to oversee. the founders can't enrich themselves without also enriching the investors. [12]
why do founders want to take the vcs' money? growth, again. the constraint between good ideas and growth operates in both directions. it's not merely that you need a scalable idea to grow. if you have such an idea and don't grow fast enough, competitors will. growing too slowly is particularly dangerous in a business with network effects, which the best startups usually have to some degree.
almost every company needs some amount of funding to get started. but startups often raise money even when they are or could be profitable. it might seem foolish to sell stock in a profitable company for less than you think it will later be worth, but it's no more foolish than buying insurance. fundamentally that's how the most successful startups view fundraising. they could grow the company on its own revenues, but the extra money and help supplied by vcs will let them grow even faster. raising money lets you *choose* your growth rate.
money to grow faster is always at the command of the most successful startups, because the vcs need them more than they need the vcs. a profitable startup could if it wanted just grow on its own revenues. growing slower

might be slightly dangerous, but chances are it wouldn't kill them. whereas vcs need to invest in startups, and in particular the most successful startups, or they'll be out of business. which means that any sufficiently promising startup will be offered money on terms they'd be crazy to refuse. and yet because of the scale of the successes in the startup business, vcs can still make money from such investments. you'd have to be crazy to believe your company was going to become as valuable as a high growth rate can make it, but some do.

pretty much every successful startup will get acquisition offers too. why? what is it about startups that makes other companies want to buy them? [13] fundamentally the same thing that makes everyone else want the stock of successful startups: a rapidly growing company is valuable. it's a good thing ebay bought paypal, for example, because paypal is now responsible for 43% of their sales and probably more of their growth.

but acquirers have an additional reason to want startups. a rapidly growing company is not merely valuable, but dangerous. if it keeps expanding, it might expand into the acquirer's own territory. most product acquisitions have some component of fear. even if an acquirer isn't threatened by the startup itself, they might be alarmed at the thought of what a competitor could do with it. and because startups are in this sense doubly valuable to acquirers, acquirers will often pay more than an ordinary investor would. [14]

understand

the combination of founders, investors, and acquirers forms a natural ecosystem. it works so well that those who don't understand it are driven to invent conspiracy theories to explain how neatly things sometimes turn out. just as our ancestors did to explain the apparently too neat workings of the natural world. but there is no secret cabal making it all work.

if you start from the mistaken assumption that instagram was worthless, you have to invent a secret boss to force mark zuckerberg to buy it. to anyone who knows mark zuckerberg, that is the reductio ad absurdum of the

initial assumption. the reason he bought instagram was that it was valuable and dangerous, and what made it so was growth.

if you want to understand startups, understand growth. growth drives everything in this world. growth is why startups usually work on technology — because ideas for fast growing companies are so rare that the best way to find new ones is to discover those recently made viable by change, and technology is the best source of rapid change. growth is why it's a rational choice economically for so many founders to try starting a startup: growth makes the successful companies so valuable that the expected value is high even though the risk is too. growth is why vcs want to invest in startups: not just because the returns are high but also because generating returns from capital gains is easier to manage than generating returns from dividends. growth explains why the most successful startups take vc money even if they don't need to: it lets them choose their growth rate. and growth explains why successful startups almost invariably get acquisition offers. to acquirers a fast-growing company is not merely valuable but dangerous too.

it's not just that if you want to succeed in some domain, you have to understand the forces driving it. understanding growth is what starting a startup *consists* of. what you're really doing (and to the dismay of some observers, all you're really doing) when you start a startup is committing to solve a harder type of problem than ordinary businesses do. you're committing to search for one of the rare ideas that generates rapid growth. because these ideas are so valuable, finding one is hard. the startup is the embodiment of your discoveries so far. starting a startup is thus very much like deciding to be a research scientist: you're not committing to solve any specific problem; you don't know for sure which problems are soluble; but you're committing to try to discover something no one knew before. a startup founder is in effect an economic research scientist. most don't discover anything that remarkable, but some discover relativity.

notes

1 strictly speaking it's not lots of customers you need but a big market, meaning a high product of number of customers times how much they'll pay. but it's dangerous to have too few customers even if they pay a lot, or the power that individual customers have over you could turn you into a de facto consulting firm. so whatever market you're in, you'll usually do best to err on the side of making the broadest type of product for it.

2 one year at startup school david heinemeier hansson encouraged programmers who wanted to start businesses to use a restaurant as a model. what he meant, i believe, is that it's fine to start software companies constrained in (a) in the same way a restaurant is constrained in (b). i agree. most people should not try to start startups.

3 that sort of stepping back is one of the things we focus on at y combinator. it's common for founders to have discovered something intuitively without understanding all its implications. that's probably true of the biggest discoveries in any field.

4 i got it wrong in "how to make wealth" when i said that a startup was a small company that takes on a hard technical problem. that is the most common recipe but not the only one.

5 in principle companies aren't limited by the size of the markets they serve, because they could just expand into new markets. but there seem to be limits on the ability of big companies to do that. which means the slowdown that comes from bumping up against the limits of one's markets is ultimately just another way in which internal limits are expressed.

it may be that some of these limits could be overcome by changing the shape of the organization — specifically by sharding it.

6 this is, obviously, only for startups that have already launched or can launch during yc. a startup building a new database will probably not do that. on the other hand, launching something small and then using growth rate as evolutionary pressure is such a valuable technique that any company that could start this way probably should.

7 if the startup is taking the facebook/twitter route and building something they hope will be very popular but from which they don't yet have a definite plan to make money, the growth rate has to be higher, even though it's a proxy for revenue growth, because such companies need huge numbers of users to succeed at all.

beware too of the edge case where something spreads rapidly but the churn is high as well, so that you have good net growth till you run through all the potential users, at which point it suddenly stops.

8 within yc when we say it's ipso facto right to do whatever gets you growth, it's implicit that this excludes trickery like buying users for more than their lifetime value, counting users as active when they're really not, bleeding out invites at a regularly increasing rate to manufacture a perfect growth curve, etc. even if you were able to fool investors with such tricks, you'd ultimately be hurting yourself, because you're throwing off your own compass.

9 which is why it's such a dangerous mistake to believe that successful startups are simply the embodiment of some brilliant initial idea. what you're looking for initially is not so much a great idea as an idea that could evolve into a great one. the danger is that promising ideas are not merely blurry versions of great ones. they're often different in kind, because the early adopters you evolve the idea upon have different needs from the rest of the market. for example, the idea that evolves into facebook isn't merely a subset of facebook; the idea that evolves into facebook is a site for harvard undergrads.

10 what if a company grew at 1.7x a year for a really long time? could it not grow just as big as any successful startup? in principle yes, of course. if our hypothetical company making $1000 a month grew at 1% a week for 19 years, it would grow as big as a company growing at 5% a week for 4 years. but while such trajectories may be common in, say, real estate development, you don't see them much in the technology business. in technology, companies that grow slowly tend not to grow as big.

11 any expected value calculation varies from person to person depending on their utility function for money. i.e. the first million is worth more to most people than subsequent millions. how much more depends on the person. for founders who are younger or more ambitious the utility function is flatter. which is probably part of the reason the founders of the most successful startups of all tend to be on the young side.

12 more precisely, this is the case in the biggest winners, which is where all the returns come from. a startup founder could pull the same trick of enriching himself at the company's expense by selling them overpriced components. but it wouldn't be worth it for the founders of google to do that. only founders of failing startups would even be tempted, but those are writeoffs from the vcs' point of view anyway.

13 acquisitions fall into two categories: those where the acquirer wants the business, and those where the acquirer just wants the employees. the latter type is sometimes called an hr acquisition. though nominally acquisitions and sometimes on a scale that has a significant effect on the expected value calculation for potential founders, hr acquisitions are viewed by acquirers as more akin to hiring bonuses.

14 i once explained this to some founders who had recently arrived from russia. they found it novel that if you threatened a company they'd pay a premium for you. "in russia they just kill you," they said, and they were only partly joking. economically, the fact that established companies can't simply eliminate new competitors may be one of the most valuable aspects of the rule of law. and so to the extent we see incumbents suppressing competitors via regulations or patent suits, we should worry, not because it's a departure from the rule of law per se but from what the rule of law is aiming at.

thanks to sam altman, marc andreessen, paul buchheit, patrick collison, jessica livingston, geoff ralston, and harj taggar for reading drafts of this.

07

before the startup

paulgraham.com/before.html

october 2014

(this essay is derived from a guest lecture in sam altman's startup class at stanford. it's intended for college students, but much of it is applicable to potential founders at other ages.)

one of the advantages of having kids is that when you have to give advice, you can ask yourself "what would i tell my own kids?" my kids are little, but i can imagine what i'd tell them about startups if they were in college, and that's what i'm going to tell you.

startups are very counterintuitive. i'm not sure why. maybe it's just because knowledge about them hasn't permeated our culture yet. but whatever the reason, starting a startup is a task where you can't always trust your instincts.

it's like skiing in that way. when you first try skiing and you want to slow down, your instinct is to lean back. but if you lean back on skis you fly down the hill out of control. so part of learning to ski is learning to suppress that impulse. eventually you get new habits, but at first it takes a conscious effort. at first there's a list of things you're trying to remember as you start down the hill.

startups are as unnatural as skiing, so there's a similar list for startups. here i'm going to give you the first part of it — the things to remember if you want to prepare yourself to start a startup.

counterintuitive

the first item on it is the fact i already mentioned: that startups are so weird that if you trust your instincts, you'll make a lot of mistakes. if you know nothing more than this, you may at least pause before making them. when i was running y combinator i used to joke that our function was to tell founders things they would ignore. it's really true. batch after batch, the yc partners warn founders about mistakes they're about to make, and the founders ignore them, and then come back a year later and say "i wish we'd listened."

why do the founders ignore the partners' advice? well, that's the thing about counterintuitive ideas: they contradict your intuitions. they seem wrong. so of course your first impulse is to disregard them. and in fact my joking description is not merely the curse of y combinator but part of its raison d'etre. if founders' instincts already gave them the right answers, they wouldn't need us. you only need other people to give you advice that surprises you. that's why there are a lot of ski instructors and not many running instructors. [1]

you can, however, trust your instincts about people. and in fact one of the most common mistakes young founders make is not to do that enough. they get involved with people who seem impressive, but about whom they feel some misgivings personally. later when things blow up they say "i knew there was something off about him, but i ignored it because he seemed so impressive."

if you're thinking about getting involved with someone — as a cofounder, an employee, an investor, or an acquirer — and you have misgivings about them, trust your gut. if someone seems slippery, or bogus, or a jerk, don't ignore it.

this is one case where it pays to be self-indulgent. work with people you genuinely like, and you've known long enough to be sure.

expertise

the second counterintuitive point is that it's not that important to know a lot about startups. the way to succeed in a startup is not to be an expert on startups, but to be an expert on your users and the problem you're solving

for them. mark zuckerberg didn't succeed because he was an expert on startups. he succeeded despite being a complete noob at startups, because he understood his users really well.

if you don't know anything about, say, how to raise an angel round, don't feel bad on that account. that sort of thing you can learn when you need to, and forget after you've done it.

in fact, i worry it's not merely unnecessary to learn in great detail about the mechanics of startups, but possibly somewhat dangerous. if i met an undergrad who knew all about convertible notes and employee agreements and (god forbid) class ff stock, i wouldn't think "here is someone who is way ahead of their peers." it would set off alarms. because another of the characteristic mistakes of young founders is to go through the motions of starting a startup. they make up some plausible-sounding idea, raise money at a good valuation, rent a cool office, hire a bunch of people. from the outside that seems like what startups do. but the next step after rent a cool office and hire a bunch of people is: gradually realize how completely fucked they are, because while imitating all the outward forms of a startup they have neglected the one thing that's actually essential: making something people want.

game

we saw this happen so often that we made up a name for it: playing house. eventually i realized why it was happening. the reason young founders go through the motions of starting a startup is because that's what they've been trained to do for their whole lives up to that point. think about what you have to do to get into college, for example. extracurricular activities, check. even in college classes most of the work is as artificial as running laps.

i'm not attacking the educational system for being this way. there will always be a certain amount of fakeness in the work you do when you're being taught something, and if you measure their performance it's inevitable that people will exploit the difference to the point where much of what you're measuring is artifacts of the fakeness.

i confess i did it myself in college. i found that in a lot of classes there might only be 20 or 30 ideas that were the right shape to make good exam questions. the way i studied for exams in these classes was not (except incidentally) to master the material taught in the class, but to make a list of potential exam questions and work out the answers in advance. when i walked into the final, the main thing i'd be feeling was curiosity about which of my questions would turn up on the exam. it was like a game.

it's not surprising that after being trained for their whole lives to play such games, young founders' first impulse on starting a startup is to try to figure out the tricks for winning at this new game. since fundraising appears to be the measure of success for startups (another classic noob mistake), they always want to know what the tricks are for convincing investors. we tell them the best way to convince investors is to make a startup that's actually doing well, meaning growing fast, and then simply tell investors so. then they want to know what the tricks are for growing fast. and we have to tell them the best way to do that is simply to make something people want.

so many of the conversations yc partners have with young founders begin with the founder asking "how do we..." and the partner replying "just..." why do the founders always make things so complicated? the reason, i realized, is that they're looking for the trick.

so this is the third counterintuitive thing to remember about startups: starting a startup is where gaming the system stops working. gaming the system may continue to work if you go to work for a big company. depending on how broken the company is, you can succeed by sucking up to the right people, giving the impression of productivity, and so on. [2] but that doesn't work with startups. there is no boss to trick, only users, and all users care about is whether your product does what they want. startups are as impersonal as physics. you have to make something people want, and you prosper only to the extent you do.

the dangerous thing is, faking does work to some degree on investors. if you're super good at sounding like you know what you're talking about, you can fool investors for at least one and perhaps even two rounds of

funding. but it's not in your interest to. the company is ultimately doomed. all you're doing is wasting your own time riding it down.

so stop looking for the trick. there are tricks in startups, as there are in any domain, but they are an order of magnitude less important than solving the real problem. a founder who knows nothing about fundraising but has made something users love will have an easier time raising money than one who knows every trick in the book but has a flat usage graph. and more importantly, the founder who has made something users love is the one who will go on to succeed after raising the money.

though in a sense it's bad news in that you're deprived of one of your most powerful weapons, i think it's exciting that gaming the system stops working when you start a startup. it's exciting that there even exist parts of the world where you win by doing good work. imagine how depressing the world would be if it were all like school and big companies, where you either have to spend a lot of time on bullshit things or lose to people who do. [3] i would have been delighted if i'd realized in college that there were parts of the real world where gaming the system mattered less than others, and a few where it hardly mattered at all. but there are, and this variation is one of the most important things to consider when you're thinking about your future. how do you win in each type of work, and what would you like to win by doing? [4]

all-consuming

that brings us to our fourth counterintuitive point: startups are all-consuming. if you start a startup, it will take over your life to a degree you cannot imagine. and if your startup succeeds, it will take over your life for a long time: for several years at the very least, maybe for a decade, maybe for the rest of your working life. so there is a real opportunity cost here.

larry page may seem to have an enviable life, but there are aspects of it that are unenviable. basically at 25 he started running as fast as he could and it must seem to him that he hasn't stopped to catch his breath since. every day new shit happens in the google empire that only the ceo can deal with, and he, as ceo, has to deal with it. if he goes on vacation for even a

week, a whole week's backlog of shit accumulates. and he has to bear this uncomplainingly, partly because as the company's daddy he can never show fear or weakness, and partly because billionaires get less than zero sympathy if they talk about having difficult lives. which has the strange side effect that the difficulty of being a successful startup founder is concealed from almost everyone except those who've done it.

y combinator has now funded several companies that can be called big successes, and in every single case the founders say the same thing. it never gets any easier. the nature of the problems change. you're worrying about construction delays at your london office instead of the broken air conditioner in your studio apartment. but the total volume of worry never decreases; if anything it increases.

starting a successful startup is similar to having kids in that it's like a button you push that changes your life irrevocably. and while it's truly wonderful having kids, there are a lot of things that are easier to do before you have them than after. many of which will make you a better parent when you do have kids. and since you can delay pushing the button for a while, most people in rich countries do.

yet when it comes to startups, a lot of people seem to think they're supposed to start them while they're still in college. are you crazy? and what are the universities thinking? they go out of their way to ensure their students are well supplied with contraceptives, and yet they're setting up entrepreneurship programs and startup incubators left and right.

to be fair, the universities have their hand forced here. a lot of incoming students are interested in startups. universities are, at least de facto, expected to prepare them for their careers. so students who want to start startups hope universities can teach them about startups. and whether universities can do this or not, there's some pressure to claim they can, lest they lose applicants to other universities that do.

can universities teach students about startups? yes and no. they can teach students about startups, but as i explained before, this is not what you need to know. what you need to learn about are the needs of your own

users, and you can't do that until you actually start the company. [5] so starting a startup is intrinsically something you can only really learn by doing it. and it's impossible to do that in college, for the reason i just explained: startups take over your life. you can't start a startup for real as a student, because if you start a startup for real you're not a student anymore. you may be nominally a student for a bit, but you won't even be that for long. [6] given this dichotomy, which of the two paths should you take? be a real student and not start a startup, or start a real startup and not be a student? i can answer that one for you. do not start a startup in college. how to start a startup is just a subset of a bigger problem you're trying to solve: how to have a good life. and though starting a startup can be part of a good life for a lot of ambitious people, age 20 is not the optimal time to do it. starting a startup is like a brutally fast depth-first search. most people should still be searching breadth-first at 20.

you can do things in your early 20s that you can't do as well before or after, like plunge deeply into projects on a whim and travel super cheaply with no sense of a deadline. for unambitious people, this sort of thing is the dreaded "failure to launch," but for the ambitious ones it can be an incomparably valuable sort of exploration. if you start a startup at 20 and you're sufficiently successful, you'll never get to do it. [7]

mark zuckerberg will never get to bum around a foreign country. he can do other things most people can't, like charter jets to fly him to foreign countries. but success has taken a lot of the serendipity out of his life. facebook is running him as much as he's running facebook. and while it can be very cool to be in the grip of a project you consider your life's work, there are advantages to serendipity too, especially early in life. among other things it gives you more options to choose your life's work from.

there's not even a tradeoff here. you're not sacrificing anything if you forgo starting a startup at 20, because you're more likely to succeed if you wait. in the unlikely case that you're 20 and one of your side projects takes off like facebook did, you'll face a choice of running with it or not, and it may be reasonable to run with it. but the usual way startups take off is for the

founders to make them take off, and it's gratuitously stupid to do that at 20.

try

should you do it at any age? i realize i've made startups sound pretty hard. if i haven't, let me try again: starting a startup is really hard. what if it's too hard? how can you tell if you're up to this challenge?

the answer is the fifth counterintuitive point: you can't tell. your life so far may have given you some idea what your prospects might be if you tried to become a mathematician, or a professional football player. but unless you've had a very strange life you haven't done much that was like being a startup founder. starting a startup will change you a lot. so what you're trying to estimate is not just what you are, but what you could grow into, and who can do that?

for the past 9 years it was my job to predict whether people would have what it took to start successful startups. it was easy to tell how smart they were, and most people reading this will be over that threshold. the hard part was predicting how tough and ambitious they would become. there may be no one who has more experience at trying to predict that, so i can tell you how much an expert can know about it, and the answer is: not much. i learned to keep a completely open mind about which of the startups in each batch would turn out to be the stars.

the founders sometimes think they know. some arrive feeling sure they will ace y combinator just as they've aced every one of the (few, artificial, easy) tests they've faced in life so far. others arrive wondering how they got in, and hoping yc doesn't discover whatever mistake caused it to accept them. but there is little correlation between founders' initial attitudes and how well their companies do.

i've read that the same is true in the military — that the swaggering recruits are no more likely to turn out to be really tough than the quiet ones. and probably for the same reason: that the tests involved are so different from the ones in their previous lives.

if you're absolutely terrified of starting a startup, you probably shouldn't do it. but if you're merely unsure whether you're up to it, the only way to find out is to try. just not now.

ideas

so if you want to start a startup one day, what should you do in college? there are only two things you need initially: an idea and cofounders. and the m.o. for getting both is the same. which leads to our sixth and last counterintuitive point: that the way to get startup ideas is not to try to think of startup ideas.

i've written a whole essay on this, so i won't repeat it all here. but the short version is that if you make a conscious effort to think of startup ideas, the ideas you come up with will not merely be bad, but bad and plausible-sounding, meaning you'll waste a lot of time on them before realizing they're bad.

the way to come up with good startup ideas is to take a step back. instead of making a conscious effort to think of startup ideas, turn your mind into the type that startup ideas form in without any conscious effort. in fact, so unconsciously that you don't even realize at first that they're startup ideas. this is not only possible, it's how apple, yahoo, google, and facebook all got started. none of these companies were even meant to be companies at first. they were all just side projects. the best startups almost have to start as side projects, because great ideas tend to be such outliers that your conscious mind would reject them as ideas for companies.

ok, so how do you turn your mind into the type that startup ideas form in unconsciously? (1) learn a lot about things that matter, then (2) work on problems that interest you (3) with people you like and respect. the third part, incidentally, is how you get cofounders at the same time as the idea. the first time i wrote that paragraph, instead of "learn a lot about things that matter," i wrote "become good at some technology." but that prescription, though sufficient, is too narrow. what was special about brian chesky and joe gebbia was not that they were experts in technology. they were good at design, and perhaps even more importantly, they were good at org-

anizing groups and making projects happen. so you don't have to work on technology per se, so long as you work on problems demanding enough to stretch you.

what kind of problems are those? that is very hard to answer in the general case. history is full of examples of young people who were working on important problems that no one else at the time thought were important, and in particular that their parents didn't think were important. on the other hand, history is even fuller of examples of parents who thought their kids were wasting their time and who were right. so how do you know when you're working on real stuff? [8]

i know how *i* know. real problems are interesting, and i am self-indulgent in the sense that i always want to work on interesting things, even if no one else cares about them (in fact, especially if no one else cares about them), and find it very hard to make myself work on boring things, even if they're supposed to be important.

my life is full of case after case where i worked on something just because it seemed interesting, and it turned out later to be useful in some worldly way. y combinator itself was something i only did because it seemed interesting. so i seem to have some sort of internal compass that helps me out. but i don't know what other people have in their heads. maybe if i think more about this i can come up with heuristics for recognizing genuinely interesting problems, but for the moment the best i can offer is the hopelessly question-begging advice that if you have a taste for genuinely interesting problems, indulging it energetically is the best way to prepare yourself for a startup. and indeed, probably also the best way to live. [9]

but although i can't explain in the general case what counts as an interesting problem, i can tell you about a large subset of them. if you think of technology as something that's spreading like a sort of fractal stain, every moving point on the edge represents an interesting problem. so one guaranteed way to turn your mind into the type that has good startup ideas is to get yourself to the leading edge of some technology — to cause yourself, as paul buchheit put it, to "live in the future." when you reach that point,

ideas that will seem to other people uncannily prescient will seem obvious to you. you may not realize they're startup ideas, but you'll know they're something that ought to exist.

for example, back at harvard in the mid 90s a fellow grad student of my friends robert and trevor wrote his own voice over ip software. he didn't mean it to be a startup, and he never tried to turn it into one. he just wanted to talk to his girlfriend in taiwan without paying for long distance calls, and since he was an expert on networks it seemed obvious to him that the way to do it was turn the sound into packets and ship it over the internet. he never did any more with his software than talk to his girlfriend, but this is exactly the way the best startups get started.

so strangely enough the optimal thing to do in college if you want to be a successful startup founder is not some sort of new, vocational version of college focused on "entrepreneurship." it's the classic version of college as education for its own sake. if you want to start a startup after college, what you should do in college is learn powerful things. and if you have genuine intellectual curiosity, that's what you'll naturally tend to do if you just follow your own inclinations. [10]

the component of entrepreneurship that really matters is domain expertise. the way to become larry page was to become an expert on search. and the way to become an expert on search was to be driven by genuine curiosity, not some ulterior motive.

at its best, starting a startup is merely an ulterior motive for curiosity. and you'll do it best if you introduce the ulterior motive toward the end of the process.

so here is the ultimate advice for young would-be startup founders, boiled down to two words: just learn.

notes

1 some founders listen more than others, and this tends to be a predictor of success. one of the things i remember about the airbnbs during yc is how intently they listened.

2 in fact, this is one of the reasons startups are possible. if big companies weren't plagued by internal inefficiencies, they'd be proportionately more effective, leaving less room for startups.

3 in a startup you have to spend a lot of time on schleps, but this sort of work is merely unglamorous, not bogus.

4 what should you do if your true calling is gaming the system? management consulting.

5 the company may not be incorporated, but if you start to get significant numbers of users, you've started it, whether you realize it yet or not.

6 it shouldn't be that surprising that colleges can't teach students how to be good startup founders, because they can't teach them how to be good employees either.

the way universities "teach" students how to be employees is to hand off the task to companies via internship programs. but you couldn't do the equivalent thing for startups, because by definition if the students did well they would never come back.

7 charles darwin was 22 when he received an invitation to travel aboard the hms beagle as a naturalist. it was only because he was otherwise unoccupied, to a degree that alarmed his family, that he could accept it. and yet if he hadn't we probably would not know his name.

8 parents can sometimes be especially conservative in this department. there are some whose definition of important problems includes only those on the critical path to med school.

9 i did manage to think of a heuristic for detecting whether you have a taste for interesting ideas: whether you find known boring ideas intolerable. could you endure studying literary theory, or working in middle management at a large company?

10 in fact, if your goal is to start a startup, you can stick even more closely to the ideal of a liberal education than past generations have. back when students focused mainly on getting a job after college, they thought at least a little about how the courses they took might look to an employer. and perhaps even worse, they might shy away from taking a difficult class lest they get a low grade, which would harm their all-important gpa. good news: users don't care what your gpa was. and i've never heard of investors caring either. y combinator certainly never asks what classes you took in college or what grades you got in them.

thanks to sam altman, paul buchheit, john collison, patrick collison, jessica livingston, robert morris, geoff ralston, and fred wilson for reading drafts of this.

08

startups in 13 sentences

paulgraham.com/13sentences.html

watch how this essay was written.

february 2009
one of the things i always tell startups is a principle i learned from paul buchheit: it's better to make a few people really happy than to make a lot of people semi-happy. i was saying recently to a reporter that if i could only tell startups 10 things, this would be one of them. then i thought: what would the other 9 be?
when i made the list there turned out to be 13:
1. pick good cofounders.
cofounders are for a startup what location is for real estate. you can change anything about a house except where it is. in a startup you can change your idea easily, but changing your cofounders is hard. [1] and the success of a startup is almost always a function of its founders.
2. launch fast.
the reason to launch fast is not so much that it's critical to get your product to market early, but that you haven't really started working on it till you've launched. launching teaches you what you should have been building. till you know that you're wasting your time. so the main value of whatever you launch with is as a pretext for engaging users.
3. let your idea evolve.

this is the second half of launching fast. launch fast and iterate. it's a big mistake to treat a startup as if it were merely a matter of implementing some brilliant initial idea. as in an essay, most of the ideas appear in the implementing.

4. understand your users.

you can envision the wealth created by a startup as a rectangle, where one side is the number of users and the other is how much you improve their lives. [2] the second dimension is the one you have most control over. and indeed, the growth in the first will be driven by how well you do in the second. as in science, the hard part is not answering questions but asking them: the hard part is seeing something new that users lack. the better you understand them the better the odds of doing that. that's why so many successful startups make something the founders needed.

5. better to make a few users love you than a lot ambivalent.

ideally you want to make large numbers of users love you, but you can't expect to hit that right away. initially you have to choose between satisfying all the needs of a subset of potential users, or satisfying a subset of the needs of all potential users. take the first. it's easier to expand userwise than satisfactionwise. and perhaps more importantly, it's harder to lie to yourself. if you think you're 85% of the way to a great product, how do you know it's not 70%? or 10%? whereas it's easy to know how many users you have.

6. offer surprisingly good customer service.

customers are used to being maltreated. most of the companies they deal with are quasi-monopolies that get away with atrocious customer service. your own ideas about what's possible have been unconsciously lowered by such experiences. try making your customer service not merely good, but surprisingly good. go out of your way to make people happy. they'll be overwhelmed; you'll see. in the earliest stages of a startup, it pays to offer customer service on a level that wouldn't scale, because it's a way of learning about your users.

7. you make what you measure.

i learned this one from joe kraus. [3] merely measuring something has an uncanny tendency to improve it. if you want to make your user numbers go up, put a big piece of paper on your wall and every day plot the number of users. you'll be delighted when it goes up and disappointed when it goes down. pretty soon you'll start noticing what makes the number go up, and you'll start to do more of that. corollary: be careful what you measure.

8. spend little.

i can't emphasize enough how important it is for a startup to be cheap. most startups fail before they make something people want, and the most common form of failure is running out of money. so being cheap is (almost) interchangeable with iterating rapidly. [4] but it's more than that. a culture of cheapness keeps companies young in something like the way exercise keeps people young.

9. get ramen profitable.

"ramen profitable" means a startup makes just enough to pay the founders' living expenses. it's not rapid prototyping for business models (though it can be), but more a way of hacking the investment process. once you cross over into ramen profitable, it completely changes your relationship with investors. it's also great for morale.

10. avoid distractions.

nothing kills startups like distractions. the worst type are those that pay money: day jobs, consulting, profitable side-projects. the startup may have more long-term potential, but you'll always interrupt working on it to answer calls from people paying you now. paradoxically, fundraising is this type of distraction, so try to minimize that too.

11. don't get demoralized.

though the immediate cause of death in a startup tends to be running out of money, the underlying cause is usually lack of focus. either the company is run by stupid people (which can't be fixed with advice) or the people are smart but got demoralized. starting a startup is a huge moral weight. understand this and make a conscious effort not to be ground down by it, just as you'd be careful to bend at the knees when picking up a heavy box.

12. don't give up.
even if you get demoralized, don't give up. you can get surprisingly far by just not giving up. this isn't true in all fields. there are a lot of people who couldn't become good mathematicians no matter how long they persisted. but startups aren't like that. sheer effort is usually enough, so long as you keep morphing your idea.
13. deals fall through.
one of the most useful skills we learned from viaweb was not getting our hopes up. we probably had 20 deals of various types fall through. after the first 10 or so we learned to treat deals as background processes that we should ignore till they terminated. it's very dangerous to morale to start to depend on deals closing, not just because they so often don't, but because it makes them less likely to.
having gotten it down to 13 sentences, i asked myself which i'd choose if i could only keep one.
understand your users. that's the key. the essential task in a startup is to create wealth; the dimension of wealth you have most control over is how much you improve users' lives; and the hardest part of that is knowing what to make for them. once you know what to make, it's mere effort to make it, and most decent hackers are capable of that.
understanding your users is part of half the principles in this list. that's the reason to launch early, to understand your users. evolving your idea is the embodiment of understanding your users. understanding your users well will tend to push you toward making something that makes a few people deeply happy. the most important reason for having surprisingly good customer service is that it helps you understand your users. and understanding your users will even ensure your morale, because when everything else is collapsing around you, having just ten users who love you will keep you going.

notes

1 strictly speaking it's impossible without a time machine.

2 in practice it's more like a ragged comb.

3 joe thinks one of the founders of hewlett packard said it first, but he doesn't remember which.

4 they'd be interchangeable if markets stood still. since they don't, working twice as fast is better than having twice as much time.

09

the reddits

march 2024
paulgraham.com/reddits.html

i met the reddits before we even started y combinator. in fact they were one of the reasons we started it.

yc grew out of a talk i gave to the harvard computer society (the undergrad computer club) about how to start a startup. everyone else in the audience was probably local, but steve and alexis came up on the train from the university of virginia, where they were seniors. since they'd come so far i agreed to meet them for coffee. they told me about the startup idea we'd later fund them to drop: a way to order fast food on your cellphone.

this was before smartphones. they'd have had to make deals with cell carriers and fast food chains just to get it launched. so it was not going to happen. it still doesn't exist, 19 years later. but i was impressed with their brains and their energy. in fact i was so impressed with them and some of the other people i met at that talk that i decided to start something to fund them. a few days later i told steve and alexis that we were starting y combinator, and encouraged them to apply.

that first batch we didn't have any way to identify applicants, so we made up nicknames for them. the reddits were the "cell food muffins." "muffin" is a term of endearment jessica uses for things like small dogs and two year olds. so that gives you some idea what kind of impression steve and alexis made in those days. they had the look of slightly ruffled surprise that baby birds have.

their idea was bad though. and since we thought then that we were funding ideas rather than founders, we rejected them. but we felt bad about it.

jessica was sad that we'd rejected the muffins. and it seemed wrong to me to turn down the people we'd been inspired to start yc to fund.

i don't think the startup sense of the word "pivot" had been invented yet, but we wanted to fund steve and alexis, so if their idea was bad, they'd have to work on something else. and i knew what else. in those days there was a site called delicious where you could save links. it had a page called del.icio.us/popular that listed the most-saved links, and people were using this page as a de facto reddit. i knew because a lot of the traffic to my site was coming from it. there needed to be something like del.icio.us/popular, but designed for sharing links instead of being a byproduct of saving them. so i called steve and alexis and said that we liked them, just not their idea, so we'd fund them if they'd work on something else. they were on the train home to virginia at that point. they got off at the next station and got on the next train north, and by the end of the day were committed to working on what's now called reddit.

they would have liked to call it snoo, as in "what snoo?" but snoo.com was too expensive, so they settled for calling the mascot snoo and picked a name for the site that wasn't registered. early on reddit was just a provisional name, or so they told me at least, but it's probably too late to change it now.

as with all the really great startups, there's an uncannily close match between the company and the founders. steve in particular. reddit has a certain personality — curious, skeptical, ready to be amused — and that personality is steve's.

steve will roll his eyes at this, but he's an intellectual; he's interested in ideas for their own sake. that was how he came to be in that audience in cambridge in the first place. he knew me because he was interested in a programming language i've written about called lisp, and lisp is one of those languages few people learn except out of intellectual curiosity. steve's kind of vacuum-cleaner curiosity is exactly what you want when you're starting a site that's a list of links to literally anything interesting.

steve was not a big fan of authority, so he also liked the idea of a site without editors. in those days the top forum for programmers was a site called slashdot. it was a lot like reddit, except the stories on the frontpage were chosen by human moderators. and though they did a good job, that one small difference turned out to be a big difference. being driven by user submissions meant reddit was fresher than slashdot. news there was newer, and users will always go where the newest news is.

i pushed the reddits to launch fast. a version one didn't need to be more than a couple hundred lines of code. how could that take more than a week or two to build? and they did launch comparatively fast, about three weeks into the first yc batch. the first users were steve, alexis, me, and some of their yc batchmates and college friends. it turns out you don't need that many users to collect a decent list of interesting links, especially if you have multiple accounts per user.

reddit got two more people from their yc batch: chris slowe and aaron swartz, and they too were unusually smart. chris was just finishing his phd in physics at harvard. aaron was younger, a college freshman, and even more anti-authority than steve. it's not exaggerating to describe him as a martyr for what authority later did to him.

slowly but inexorably reddit's traffic grew. at first the numbers were so small they were hard to distinguish from background noise. but within a few weeks it was clear that there was a core of real users returning regularly to the site. and although all kinds of things have happened to reddit the company in the years since, reddit the *site* never looked back.

reddit the site (and now app) is such a fundamentally useful thing that it's almost unkillable. which is why, despite a long stretch after steve left when the management strategy ranged from benign neglect to spectacular blunders, traffic just kept growing. you can't do that with most companies. most companies you take your eye off the ball for six months and you're in deep trouble. but reddit was special, and when steve came back in 2015, i knew the world was in for a surprise.

people thought they had reddit's number: one of the players in silicon valley, but not one of the big ones. but those who knew what had been going on behind the scenes knew there was more to the story than this. if reddit could grow to the size it had with management that was harmless at best, what could it do if steve came back? we now know the answer to that question. or at least a lower bound on the answer. steve is not out of ideas yet.

10

how to start google

march 2024
paulgraham.com/google.html

(this is a talk i gave to 14 and 15 year olds about what to do now if they might want to start a startup later. lots of schools think they should tell students something about startups. this is what i think they should tell them.)
most of you probably think that when you're released into the so-called real world you'll eventually have to get some kind of job. that's not true, and today i'm going to talk about a trick you can use to avoid ever having to get a job.
the trick is to start your own company. so it's not a trick for avoiding *work*, because if you start your own company you'll work harder than you would if you had an ordinary job. but you will avoid many of the annoying things that come with a job, including a boss telling you what to do.
it's more exciting to work on your own project than someone else's. and you can also get a lot richer. in fact, this is the standard way to get really rich. if you look at the lists of the richest people that occasionally get published in the press, nearly all of them did it by starting their own companies.
starting your own company can mean anything from starting a barber shop to starting google. i'm here to talk about one extreme end of that continuum. i'm going to tell you how to start google.
the companies at the google end of the continuum are called startups when they're young. the reason i know about them is that my wife jessica and i started something called y combinator that is basically a startup factory. since 2005, y combinator has funded over 4000 startups. so we know exa-

ctly what you need to start a startup, because we've helped people do it for the last 19 years.

you might have thought i was joking when i said i was going to tell you how to start google. you might be thinking "how could *we* start google?" but that's effectively what the people who did start google were thinking before they started it. if you'd told larry page and sergey brin, the founders of google, that the company they were about to start would one day be worth over a trillion dollars, their heads would have exploded.

all you can know when you start working on a startup is that it seems worth pursuing. you can't know whether it will turn into a company worth billions or one that goes out of business. so when i say i'm going to tell you how to start google, i mean i'm going to tell you how to get to the point where you can start a company that has as much chance of being google as google had of being google. [1]

how do you get from where you are now to the point where you can start a successful startup? you need three things. you need to be good at some kind of technology, you need an idea for what you're going to build, and you need cofounders to start the company with.

how do you get good at technology? and how do you choose which technology to get good at? both of those questions turn out to have the same answer: work on your own projects. don't try to guess whether gene editing or llms or rockets will turn out to be the most valuable technology to know about. no one can predict that. just work on whatever interests you the most. you'll work much harder on something you're interested in than something you're doing because you think you're supposed to.

if you're not sure what technology to get good at, get good at programming. that has been the source of the median startup for the last 30 years, and this is probably not going to change in the next 10.

those of you who are taking computer science classes in school may at this point be thinking, ok, we've got this sorted. we're already being taught all about programming. but sorry, this is not enough. you have to be working on your own projects, not just learning stuff in classes. you can do well in

computer science classes without ever really learning to program. in fact you can graduate with a degree in computer science from a top university and still not be any good at programming. that's why tech companies all make you take a coding test before they'll hire you, regardless of where you went to university or how well you did there. they know grades and exam results prove nothing.

if you really want to learn to program, you have to work on your own projects. you learn so much faster that way. imagine you're writing a game and there's something you want to do in it, and you don't know how. you're going to figure out how a lot faster than you'd learn anything in a class.

you don't have to learn programming, though. if you're wondering what counts as technology, it includes practically everything you could describe using the words "make" or "build." so welding would count, or making clothes, or making videos. whatever you're most interested in. the critical distinction is whether you're producing or just consuming. are you writing computer games, or just playing them? that's the cutoff.

steve jobs, the founder of apple, spent time when he was a teenager studying calligraphy — the sort of beautiful writing that you see in medieval manuscripts. no one, including him, thought that this would help him in his career. he was just doing it because he was interested in it. but it turned out to help him a lot. the computer that made apple really big, the macintosh, came out at just the moment when computers got powerful enough to make letters like the ones in printed books instead of the computery-looking letters you see in 8 bit games. apple destroyed everyone else at this, and one reason was that steve was one of the few people in the computer business who really got graphic design.

don't feel like your projects have to be *serious*. they can be as frivolous as you like, so long as you're building things you're excited about. probably 90% of programmers start out building games. they and their friends like to play games. so they build the kind of things they and their friends want. and that's exactly what you should be doing at 15 if you want to start a startup one day.

you don't have to do just one project. in fact it's good to learn about multiple things. steve jobs didn't just learn calligraphy. he also learned about electronics, which was even more valuable. whatever you're interested in. (do you notice a theme here?)

so that's the first of the three things you need, to get good at some kind or kinds of technology. you do it the same way you get good at the violin or football: practice. if you start a startup at 22, and you start writing your own programs now, then by the time you start the company you'll have spent at least 7 years practicing writing code, and you can get pretty good at anything after practicing it for 7 years.

let's suppose you're 22 and you've succeeded: you're now really good at some technology. how do you get startup ideas? it might seem like that's the hard part. even if you are a good programmer, how do you get the idea to start google?

actually it's easy to get startup ideas once you're good at technology. once you're good at some technology, when you look at the world you see dotted outlines around the things that are missing. you start to be able to see both the things that are missing from the technology itself, and all the broken things that could be fixed using it, and each one of these is a potential startup.

in the town near our house there's a shop with a sign warning that the door is hard to close. the sign has been there for several years. to the people in the shop it must seem like this mysterious natural phenomenon that the door sticks, and all they can do is put up a sign warning customers about it. but any carpenter looking at this situation would think "why don't you just plane off the part that sticks?"

once you're good at programming, all the missing software in the world starts to become as obvious as a sticking door to a carpenter. i'll give you a real world example. back in the 20th century, american universities used to publish printed directories with all the students' names and contact info. when i tell you what these directories were called, you'll know which startu-

p i'm talking about. they were called facebooks, because they usually had a picture of each student next to their name.
so mark zuckerberg shows up at harvard in 2002, and the university still hasn't gotten the facebook online. each individual house has an online facebook, but there isn't one for the whole university. the university administration has been diligently having meetings about this, and will probably have solved the problem in another decade or so. most of the students don't consciously notice that anything is wrong. but mark is a programmer. he looks at this situation and thinks "well, this is stupid. i could write a program to fix this in one night. just let people upload their own photos and then combine the data into a new site for the whole university." so he does. and almost literally overnight he has thousands of users.
of course facebook was not a startup yet. it was just a... project. there's that word again. projects aren't just the best way to learn about technology. they're also the best source of startup ideas.
facebook was not unusual in this respect. apple and google also began as projects. apple wasn't meant to be a company. steve wozniak just wanted to build his own computer. it only turned into a company when steve jobs said "hey, i wonder if we could sell plans for this computer to other people." that's how apple started. they weren't even selling computers, just plans for computers. can you imagine how lame this company seemed?
ditto for google. larry and sergey weren't trying to start a company at first. they were just trying to make search better. before google, most search engines didn't try to sort the results they gave you in order of importance. if you searched for "rugby" they just gave you every web page that contained the word "rugby." and the web was so small in 1997 that this actually worked! kind of. there might only be 20 or 30 pages with the word "rugby," but the web was growing exponentially, which meant this way of doing search was becoming exponentially more broken. most users just thought, "wow, i sure have to look through a lot of search results to find what i want." door sticks. but like mark, larry and sergey were programmers. like mark, they looked at this situation and thought "well, this is stupid. some

pages about rugby matter more than others. let's figure out which those are and show them first."

it's obvious in retrospect that this was a great idea for a startup. it wasn't obvious at the time. it's never obvious. if it was obviously a good idea to start apple or google or facebook, someone else would have already done it. that's why the best startups grow out of projects that aren't meant to be startups. you're not trying to start a company. you're just following your instincts about what's interesting. and if you're young and good at technology, then your unconscious instincts about what's interesting are better than your conscious ideas about what would be a good company.

so it's critical, if you're a young founder, to build things for yourself and your friends to use. the biggest mistake young founders make is to build something for some mysterious group of other people. but if you can make something that you and your friends truly want to use — something your friends aren't just using out of loyalty to you, but would be really sad to lose if you shut it down — then you almost certainly have the germ of a good startup idea. it may not seem like a startup to you. it may not be obvious how to make money from it. but trust me, there's a way.

what you need in a startup idea, and all you need, is something your friends actually want. and those ideas aren't hard to see once you're good at technology. there are sticking doors everywhere. [2]

now for the third and final thing you need: a cofounder, or cofounders. the optimal startup has two or three founders, so you need one or two cofounders. how do you find them? can you predict what i'm going to say next? it's the same thing: projects. you find cofounders by working on projects with them. what you need in a cofounder is someone who's good at what they do and that you work well with, and the only way to judge this is to work with them on things.

at this point i'm going to tell you something you might not want to hear. it really matters to do well in your classes, even the ones that are just memorization or blathering about literature, because you need to do well in your classes to get into a good university. and if you want to start a startup you

should try to get into the best university you can, because that's where the best cofounders are. it's also where the best employees are. when larry and sergey started google, they began by just hiring all the smartest people they knew out of stanford, and this was a real advantage for them.

the empirical evidence is clear on this. if you look at where the largest numbers of successful startups come from, it's pretty much the same as the list of the most selective universities.

i don't think it's the prestigious names of these universities that cause more good startups to come out of them. nor do i think it's because the quality of the teaching is better. what's driving this is simply the difficulty of getting in. you have to be pretty smart and determined to get into mit or cambridge, so if you do manage to get in, you'll find the other students include a lot of smart and determined people. [3]

you don't have to start a startup with someone you meet at university. the founders of twitch met when they were seven. the founders of stripe, patrick and john collison, met when john was born. but universities are the main source of cofounders. and because they're where the cofounders are, they're also where the ideas are, because the best ideas grow out of projects you do with the people who become your cofounders.

so the list of what you need to do to get from here to starting a startup is quite short. you need to get good at technology, and the way to do that is to work on your own projects. and you need to do as well in school as you can, so you can get into a good university, because that's where the cofounders and the ideas are.

that's it, just two things, build stuff and do well in school.

---

notes

1 the rhetorical trick in this sentence is that the "google"s refer to different things. what i mean is: a company that has as much chance of growing as big

as google ultimately did as larry and sergey could have reasonably expected google itself would at the time they started it. but i think the original version is zippier.

2 making something for your friends isn't the only source of startup ideas. it's just the best source for young founders, who have the least knowledge of what other people want, and whose own wants are most predictive of future demand.

3 strangely enough this is particularly true in countries like the us where undergraduate admissions are done badly. us admissions departments make applicants jump through a lot of arbitrary hoops that have little to do with their intellectual ability. but the more arbitrary a test, the more it becomes a test of mere determination and resourcefulness. and those are the two most important qualities in startup founders. so us admissions departments are better at selecting founders than they would be if they were better at selecting students.

thanks to jared friedman, carolynn levy, jessica livingston, harj taggar, and garry tan for reading drafts of this.

11

founder mode

september 2024
paulgraham.com/foundermode.html

at a yc event last week brian chesky gave a talk that everyone who was there will remember. most founders i talked to afterward said it was the best they'd ever heard. ron conway, for the first time in his life, forgot to take notes. i'm not going to try to reproduce it here. instead i want to talk about a question it raised.

the theme of brian's talk was that the conventional wisdom about how to run larger companies is mistaken. as airbnb grew, well-meaning people advised him that he had to run the company in a certain way for it to scale. their advice could be optimistically summarized as "hire good people and give them room to do their jobs." he followed this advice and the results were disastrous. so he had to figure out a better way on his own, which he did partly by studying how steve jobs ran apple. so far it seems to be working. airbnb's free cash flow margin is now among the best in silicon valley.

the audience at this event included a lot of the most successful founders we've funded, and one after another said that the same thing had happened to them. they'd been given the same advice about how to run their companies as they grew, but instead of helping their companies, it had damaged them.

why was everyone telling these founders the wrong thing? that was the big mystery to me. and after mulling it over for a bit i figured out the answer: what they were being told was how to run a company you hadn't founded — how to run a company if you're merely a professional manager. but this m.o. is so much less effective that to founders it feels broken. there are

things founders can do that managers can't, and not doing them feels wrong to founders, because it is.

in effect there are two different ways to run a company: founder mode and manager mode. till now most people even in silicon valley have implicitly assumed that scaling a startup meant switching to manager mode. but we can infer the existence of another mode from the dismay of founders who've tried it, and the success of their attempts to escape from it.

there are as far as i know no books specifically about founder mode. business schools don't know it exists. all we have so far are the experiments of individual founders who've been figuring it out for themselves. but now that we know what we're looking for, we can search for it. i hope in a few years founder mode will be as well understood as manager mode. we can already guess at some of the ways it will differ.

the way managers are taught to run companies seems to be like modular design in the sense that you treat subtrees of the org chart as black boxes. you tell your direct reports what to do, and it's up to them to figure out how. but you don't get involved in the details of what they do. that would be micromanaging them, which is bad.

hire good people and give them room to do their jobs. sounds great when it's described that way, doesn't it? except in practice, judging from the report of founder after founder, what this often turns out to mean is: hire professional fakers and let them drive the company into the ground.

one theme i noticed both in brian's talk and when talking to founders afterward was the idea of being gaslit. founders feel like they're being gaslit from both sides — by the people telling them they have to run their companies like managers, and by the people working for them when they do. usually when everyone around you disagrees with you, your default assumption should be that you're mistaken. but this is one of the rare exceptions. vcs who haven't been founders themselves don't know how founders should run companies, and c-level execs, as a class, include some of the most skillful liars in the world. [1]

whatever founder mode consists of, it's pretty clear that it's going to break the principle that the ceo should engage with the company only via his or her direct reports. "skip-level" meetings will become the norm instead of a practice so unusual that there's a name for it. and once you abandon that constraint there are a huge number of permutations to choose from.

for example, steve jobs used to run an annual retreat for what he considered the 100 most important people at apple, and these were not the 100 people highest on the org chart. can you imagine the force of will it would take to do this at the average company? and yet imagine how useful such a thing could be. it could make a big company feel like a startup. steve presumably wouldn't have kept having these retreats if they didn't work. but i've never heard of another company doing this. so is it a good idea, or a bad one? we still don't know. that's how little we know about founder mode. [2]

obviously founders can't keep running a 2000 person company the way they ran it when it had 20. there's going to have to be some amount of delegation. where the borders of autonomy end up, and how sharp they are, will probably vary from company to company. they'll even vary from time to time within the same company, as managers earn trust. so founder mode will be more complicated than manager mode. but it will also work better. we already know that from the examples of individual founders groping their way toward it.

indeed, another prediction i'll make about founder mode is that once we figure out what it is, we'll find that a number of individual founders were already most of the way there — except that in doing what they did they were regarded by many as eccentric or worse. [3]

curiously enough it's an encouraging thought that we still know so little about founder mode. look at what founders have achieved already, and yet they've achieved this against a headwind of bad advice. imagine what they'll do once we can tell them how to run their companies like steve jobs instead of john sculley.

notes

1 the more diplomatic way of phrasing this statement would be to say that experienced c-level execs are often very skilled at managing up. and i don't think anyone with knowledge of this world would dispute that.

2 if the practice of having such retreats became so widespread that even mature companies dominated by politics started to do it, we could quantify the senescence of companies by the average depth on the org chart of those invited.

3 i also have another less optimistic prediction: as soon as the concept of founder mode becomes established, people will start misusing it. founders who are unable to delegate even things they should will use founder mode as the excuse. or managers who aren't founders will decide they should try to act like founders. that may even work, to some extent, but the results will be messy when it doesn't; the modular approach does at least limit the damage a bad ceo can do.

thanks to brian chesky, patrick collison, ron conway, jessica livingston, elon musk, ryan petersen, harj taggar, and garry tan for reading drafts of this.

12

how universities should prepare founders

august 2026
paulgraham.com/prepare.html

how should universities prepare students to start startups? y combinator is in the perfect position to answer this question, because we get them next. we're like grad school. and because yc has had 20 years to refine its model of what a promising founder looks like, you probably won't find a better target.

what do the yc partners look for? it's surprisingly simple. they want people who are good at building things and have a habit of doing it.

the hard part of startups is product: knowing what to build, and being able to build it. and that kind of knowledge comes from studying computer science or mechanical engineering or molecular biology, not management or finance. [1]

so the way to prepare undergraduates to become successful founders is not to give them some new curriculum focused on "entrepreneurship". it's to do what universities already do best — to teach them computer science and mechanical engineering and molecular biology. [2]

indeed, preparing students to start startups is closer to the ideal of liberal education than preparing them for almost any other kind of career. startups succeed or fail based on how much customers like the product. customers don't care what the founders studied in college. so founders are free to study whatever they want, as long as they get good at building things.

but building should be understood in a very broad sense. it doesn't mean all would-be founders have to study some form of engineering. almost any kind of expertise that could be described as building or creating could be useful. it was useful to steve jobs to have studied calligraphy, for example; it was one of the reasons apple dominated desktop publishing. so while math and science and engineering and design tend to be good bets, i would not want to draw a sharp line around them, because i can imagine other forms of building that could be useful. and of course you don't have to major in something to be good at it. mark zuckerberg was good at programming, but he was a psychology major, not a cs major.

the best way to describe what would-be founders should study is that they should seek out powerful ideas. but smart people are naturally attracted to powerful ideas anyway. so as long as departments teaching powerful ideas exist, the sort of people who'd make good founders will find them. [3]

in fact there are only two things universities need to change to be perfect at preparing founders: they need to make students feel that starting a startup is something they can do, and they need to encourage them to work on their own projects.

at the moment, the belief that it's possible to start a startup is very unevenly distributed. yc now gets so many applications that our application data is a reasonable proxy for interest in startups at different universities, and harvard alumni, for example, apply at about twice the rate of yale and princeton alumni. presumably harvard students aren't that different from yale and princeton students; the reason harvard students go on to start more startups is just that it's more customary there. which in turn implies that merely by making their students feel that starting a startup is a viable option, yale and princeton could at least double the number who do.

once a university has a culture of starting startups, you don't have to convince students that it's a viable option. new students learn that from older ones. but at a university that doesn't have much of a startup culture yet, there are things you can do to help this realization along. the most effective is probably to show students examples of people who've done it.

until you've seen some founders in real life, you tend to think that starting startups is something done by other people. seeing them pops that bubble. in fact seeing founders in real life is doubly inspiring: they seem impressive, but they also seem human. especially when they talk about the early years, when they were clueless and made lots of mistakes. so strangely enough seeing founders in real life makes being one seem simultaneously both desirable and accessible. it makes students think "i want to be like that, and i could."

how inspiring founders are to students is a function roughly of how rich and famous they are divided by how much older they are than the students. so it's not essential to bring famous billionaires to campus. founders in their mid twenties who are 3 years into a startup with a valuation of a couple hundred million will do as well; they may only be a twentieth as rich and famous, but they're twenty times easier for students to identify with.

---

it's obvious why universities that want their students to start startups need to make them believe it's a viable option. but why is it so important for students to work on their own projects?

there are four reasons. the first is simply that it's a great way, possibly the best way, to understand a subject really deeply. the excitement of creating something new is a much more powerful motivator than the fear of doing badly on an exam.

second, working on projects together is the best way for cofounders to discover one another. the most successful startups tend to have multiple founders, and the only way to tell for sure if someone will be good to work with is to work with them. apple and microsoft were just the last of many projects their founders had worked on together.

third, a startup is a project, so starting one will feel natural to someone who's used to working on projects of their own. it won't seem weird that there's no teacher or boss telling them what to do. they're used to telling themselves.

fourth, and perhaps most surprisingly, random side projects are where the best startup ideas come from. the best startup ideas tend to seem so implausible at first that anyone consciously looking for startup ideas would reject them. who'd expect to start a huge company by creating a student directory? so the way to discover the best startup ideas is not to look for startup ideas but just to work on whatever random projects seem interesting. because in fact such projects are far from random: young people who are good at building things are technological bellwethers, so any idea that seems interesting to them is disproportionately likely to lead somewhere valuable, even if they themselves don't realize it yet.

now it should be clear why the yc partners care a lot about the projects that applicants have worked on and not at all about their gpas. projects are the best source of knowledge, the best source of founding teams, and the best source of startup ideas.

but encouraging students to work on their own projects may be difficult for universities. it will mean giving the students more free time, and universities may not like to do that.

microsoft and meta have something in common that few people realize. they both got started during reading period at harvard. reading period is the gap between the end of classes and the beginning of final exams. it's called reading period because students are *supposed* to spend it preparing for exams. but reading period also turns out to have the unique combination of qualities that make it perfect for starting new projects: the students are all on campus, and they don't have anything due the next day. that latter constraint, especially, is a huge drag on the most ambitious students. merely eliminating it for a few weeks resulted in two trillion dollar companies. imagine what the us gdp would be if reading period at harvard were twice as long.

universities will tend to resist the idea of keeping students less busy with coursework. partly because administrators feel that if they want to achieve something, they have to do it by taking active measures. achieving something merely by leaving students alone is alien to their nature.

and they should be left alone. these things should be the students' own projects; the university should resist the temptation to make them official. partly because students will be more excited to work on a project that's entirely their own, and partly because many projects wouldn't survive official recognition, because they break some sort of rule. bill gates and mark zuckerberg both got in trouble with the harvard administration over projects they worked on as undergrads. bill broke university rules by bringing paul allen, who wasn't a student, into the computer lab with him to work on altair basic. zuck got in such trouble over facemash that he was put on disciplinary probation. and their cases are probably more the rule than the exception. universities have lots of rules, and novel projects are often untidy things.

right now there are students flying drones out of line of sight. turn a blind eye to it.

another reason it will be hard for universities to keep students less busy is that they'll worry that without some kind of oversight, most students will just waste whatever free time they're given. and they will! the price of giving the most energetic students room to do even better is that it leaves the least energetic ones room to do even worse. but that's a price that's worth paying, because if the most energetic students do better they could do a *lot* better, whereas the laziest students already learn so little that there's not much room for them to do worse. so giving all the students some of their time back could improve the average outcome a lot, even if it doesn't move the median. [4]

it may seem a bit excessive to change the whole schedule of the university just to encourage would-be founders. they're never going to be more than 10% of the students. and it probably would be excessive if this change only helped founders. but in fact giving the students some of their time back would help all the most energetic and ambitious ones. they'd all explore new things of one type or another if the pressure of work were relieved for even a week or two.

---

now that i've explained how universities should prepare founders, i should explain how not to. one thing universities can't do is actually teach students how to start startups. starting a startup is one of those things, like chemistry or painting, that you have to learn by doing. which means a properly run class on how to start a startup would have to be a lab class: the students would actually have to start startups. and i know exactly what a class of this type should look like, because yc is it. but yc is very different in structure from a university, and if you tried to cram it into an undergrad degree program, it would become a joke. are the students supposed to start these companies without any funding? are they supposed to run startups, which notoriously take every moment of your time when done properly, while simultaneously taking three or four other classes? and what if, despite these handicaps, some of the startups actually take off? are the students just supposed to abandon them? because it's either that or drop out. running a startup is incompatible with being a full time student. the only way to learn how to start a startup is to do it. those two statements are so obvious that they're practically truisms. and yet so many people manage to remain in denial about what they imply. you can't teach students how to start startups.

one common response to this inconvenient truth is to pretend to teach them how to start startups, for example by organizing business plan competitions. the students collaborate to come up with a startup idea, which they then pitch to simulated investors. this kind of exercise is not merely useless but positively misleading. it trains founders to think that fundraising is the essential step in starting a startup — that the core of starting a startup is to create a story that appeals to investors. as an investor, i can tell you that's not true. fundraising is merely a necessary evil. the people you need to impress are users, not investors, and the way you impress them is with prototypes, not words. the core of starting a startup is not creating a story that appeals to investors, but creating a product that appeals to users. [5]

in fact would-be founders should be doing exactly the opposite of what students do in business plan competitions. instead of thinking about startups without building anything, they should be building things without thinking about whether they'll turn into startups.

probably one of the reasons universities are tempted to organize bogus things like business plan competitions is that if they actually took the optimal measures to prepare students to start startups, it would look too quiet. imagine if a university were doing everything right. students would be getting a deep knowledge of how to build things in classes they were taking out of genuine interest, and working eagerly with their friends on side projects that had nothing to do with school. the students would graduate with exactly what predicts success in founders: the ability to build things and a habit of doing it. plus a significant number of those side projects would be incipient startups. and yet it would look to parents and prospective students as if the university wasn't doing anything. where are the classes on "entrepreneurship"? where is the innovation center?

and indeed this is another great thing about the optimal plan for preparing startup founders: it costs nothing extra. you don't have to hire any deans of entrepreneurship or build any new buildings. in fact if you do, those things will tend to drag you down; there's no need for them, so if they have any effect at all it will tend to be for the worse. if you have spare money, give it to the people teaching computer science or mechanical engineering or molecular biology. [6]

but if the optimal route looks too quiet, the solution is not to avoid it. the solution is to stand firm, knowing that you're doing the right thing, and eventually the results will speak for themselves. if you can develop an organic startup culture among your students and there are multiple students in every year who go on to start successful ones, this will soon become evident to anyone paying attention.

notes

1 should students still study computer science if ais will write most code? definitely. cs is an interesting subject in its own right and also a great way to understand problem solving in general. and even if you have ais writing all your code for you, you're still in the position of an engineering manager, and good engineering managers should be able to do the work of those working for them.

2 one reason i always put "entrepreneurship" in quotes is that it's a misleading word to use to describe starting startups. "entrepreneurship" simply means starting one's own business, and startups are a microscopically small subset of that world in which the rules are completely different. so conflating the two is asking for trouble.

3 of course all departments will claim to be teaching powerful ideas. but false claims of this type don't seem much of a danger. the sort of people who'd make good founders wouldn't even need to see through them; they simply wouldn't be interested enough in the classes taught by such departments to have much of their time wasted by them.

4 there's an interesting parallel here to variation in income. the bottom of the income scale is anchored firmly at zero, because there are some people who are either incapable of working or just not interested in doing it at the moment. if you let there be more variation in income, it won't affect the income of the people at this end of the scale; n times zero is zero; but at the other end of the scale you'll see enormous change.

5 presumably one reason these competitions lean toward impressing investors rather than users is that it's the only way to have a single set of judges. investors can be treated as interchangeable, whereas the users of each product might be different. but if it's impractical to measure the right thing, that doesn't mean the solution is to measure the wrong one.

6 another thing that will tend to draw universities away from the optimal path is business schools, if they have them. business schools were not designed to train founders. they were designed to train the managerial class of the large industri-

ustrial capitalism. that's why their official name is usually the school of management. but while the skills they teach might be useful in running companies beyond a certain size, they're not the critical ingredient in *founding* them. and the skills that are are already taught by other departments. so to the extent business schools affect their parent university's strategy for preparing founders, it can only be by adding error.

thanks to trevor blackwell, daniel diermeier, jared friedman, diana hu, michael kotlikoff, jessica livingston, robert morris, harj taggar, and garry tan for reading drafts of this.

13

how to earn a billion dollars

june 2026
paulgraham.com/earn.html

(this is based on a talk i gave at the oxford union.)
since this is apparently the future prime ministers' club, i'm going to tell you about something it would be good if more politicians understood: i'm going to tell you how people become billionaires. i hope this will be useful to you even if you don't plan to go into politics. those of you who don't become prime minister can become billionaires instead.
the reason i know about this topic is that 21 years ago jessica and i started something called y combinator. if you haven't heard of y combinator, it's a cross between an investment firm and a school for startup founders. since we started it in 2005 we've funded about 6500 companies.
starting a successful startup is the most common way to become a billionaire, so in effect i've spent the last 21 years training people to become billionaires. so far about 30 of them have, but there are many more in the pipeline.
so you can imagine how astonished i was last month when an american politician said that it was impossible to earn a billion dollars. i felt like a skating coach hearing someone say that it's impossible to do a triple axel. of course it's possible. it's *hard*, but it's possible.
she wasn't saying, of course, that it's impossible to become a billionaire. obviously that's possible. nor was she talking about the distinction between income and capital gains. she wasn't making a point about accounting. what she meant was that it's impossible to get that rich without doing something bad — without cheating in some way.

a couple days later i was talking to the founder of a startup i'd funded. i began by asking, as i usually do when i meet a founder, what her growth rate was. 93% last month, she said. i pointed out that this meant her net worth was also growing at 93% a month. she was getting richer at a stupendously rapid rate. and yet she hadn't been doing anything bad. the reason her startup was growing so fast was simply that users loved what she'd built. so she could feel from her own experience how wrong that politician was. she wasn't exploiting anyone. exactly the opposite in fact. the reason her startup was growing so fast was that she and her cofounder had been working their asses off to make their users happy, and as a result the users had been telling their friends. and that gets you exponential growth.

later that day i was talking about her case online, and someone replied that having a few million and growing at 93% a month was radically different from being a billionaire.

i suspect many people would agree with this statement. but it turns out not merely to be false, but false in a very illuminating way.

so i would like you all to do me a favor please. i would like you to take out your phones and calculate a number. i know this may seem contrived, but i promise it will be useful for you. i'm going to have you do the most common kind of calculation i do as an investor, and the experience will bring home to you what startups are all about.

if we interpret his statement in the most conservative way and assume that a few means 2, her company has to grow 500x for her to become a billionaire. so we are going to calculate how many months of 93% growth it takes for something to grow 500x.

to do this we want to calculate the log base 1.93 of 500. the easiest way to do that is to go to google search, which lets you do calculations right in the search box. so go to google search and type log(500, 1.93). if you typed that right, the answer you get is about 9.45.

that is how many months of 93% growth it takes to become a billionaire, starting from 2 million. a couple million and 93% growth are not, in fact, radically different from a billion. they're nine and a half months apart.

now you see why, when i meet a founder, the first thing i ask about is their growth rate.

but i don't want anyone to accuse me of using unrealistic numbers, so let's take a more conservative growth rate. let's see what happens at 15% a month. that's not rare at all. i constantly encounter startups growing at 15% a month.

if your revenues grow at 15% a month, how much more will you be making 5 years from now? to calculate that, we need to find 1.15 to the 60th power (since 5 years is 60 months). so go to google again and this time type 1.15 ^ 60. the answer should be about 4384. meaning in 5 years your startup will be making 4384 times as much. if you're currently making ten thousand a month, in five years you'll be making about 44 million a month, or 526 million a year. and at that point, if you own as much of the company as founders typically do, you will be a billionaire.

in the real world, growth rates tend to slow down a bit. a very successful startup will probably be growing faster than 15% a month in year 1 and slower than 15% a month in year 5. but you end up in about the same place. if you start a startup in your early twenties, it's definitely possible to be a billionaire by the time you're thirty. hard, but possible.

i wanted you to feel this by doing the calculation yourselves, because now you understand one of the reasons people start startups. exponential growth is like magic. it generates outcomes that seem impossible. and that's why some politicians distrust it. they don't understand the math of exponential growth, so when they see people becoming what seems to them impossibly rich, they assume they must have cheated.

but now you at least understand, from having done the math yourselves, that you don't have to cheat to become a billionaire. you've seen for yourselves that there are only two numbers in the calculation, the growth rate and how long it continues. if it's impossible to make a billion dollars without cheating, which of those two numbers is impossible? it's certainly not impossible to grow at 15% a month without cheating. startups do that all the time. and how long you can continue to grow at that rate depends on

the size of the market. obviously for you to grow 4000x, there has to be at least 4000x more demand. but that's all you need. and how could you possibly cheat to increase the market size?

if you're only planning to become prime minister, you can stop paying attention now. we've proved that it is in fact possible to earn a billion dollars, because it only depends on two numbers, one of which startups routinely hit without cheating, and another that cheating couldn't possibly affect.

but if you actually want to become a billionaire, we should go into more detail. especially about that first number, the growth rate. to grow at a consistent rate every month, you have to make something so good that people tell their friends about it. and in fact that's the other reason i always begin by asking founders their growth rate. it shows whether they've built the right thing.

so how, exactly, do you make something people like so much that they tell their friends about it? the problem with market economies, and also the great thing about market economies, is that it's hard to make something customers want that they don't already have. as soon as a new, satisfiable need is discovered, people rush to satisfy it. so you're going to have to discover a need that no one else knows about yet.

how do you do that? *by feeling the need yourself.*

you're young, and usually young founders should make something that they themselves want. you don't have enough experience yet to know what other people need. but at the same time your own needs are uniquely valuable, because your needs predict future demand. you're the age when people start using new things. whatever you and your friends start using now, everyone is going to be using in ten years. since your intuitions about other people's needs are usually a crap signal, and your own needs are an especially valuable one, you should usually listen to the second signal; you should make something you and your friends want.

making something you and your friends want doesn't mean you have to build a consumer product. maybe you and your friends are molecular biologists, and there's something cool that could be done now to dna that

everyone else has overlooked. maybe you and your friends are into drones. the idea doesn't have to have a wide appeal. it literally just has to appeal to you and your friends.

don't worry about the second number, the market size. since you predict future demand, the market will grow. and it's always possible to expand into adjacent markets. all you need is a beachhead in the territory of unsatisfied need that you can expand from.

how do you get an idea like that? the answer is one of the most counterintuitive things about startups — which is saying something, because there are a lot of counterintuitive things about startups. but the way to get the very best startup ideas is not to look for startup ideas. if you're consciously looking for startup ideas, it will make you too conservative. you'll lop off the outliers. because the very best startup ideas tend to sound so lame, at first, that you'd reject them if you were consciously looking for startup ideas. that's what has prevented them from being discovered.

imagine what a bad idea apple or facebook or airbnb seemed at first. how many people are going to want their own computers? how is a company going to make money from undergrads stalking one another online? who's going to pay to sleep on an airbed on someone's floor? we know how these ideas turned out, so it's easy to rewrite history, but i remember very well how bad facebook and airbnb sounded at first. we *funded* airbnb, and we thought the idea was bad. the reason we funded them was just that we liked the founders.

so how do you find startup ideas without looking for them? by working on projects with your friends. that's where the very best startups come from. initially they're not even meant to be companies. they're just something people built because they thought it would be cool. that's how apple and google and facebook all started. none of them were meant to be companies at first.

the reason this works is what i told you earlier: you predict future demand. so if you just build random stuff you think would be cool, the things you build will actually be far from random.

this is one of those cases where your unconscious mind knows more than your conscious mind does. anything that genuinely seems to you like it would be a cool thing to build has a high probability of leading to a good startup idea, no matter how preposterous it sounds. whatever you build couldn't possibly sound more preposterous than a startup we funded in 2006 called justin.tv. it consisted of one guy, justin kan, walking around with a camera on the side of his head, live streaming everything that happened to him. but this company ended up doing quite well. in fact you've probably heard of it, but under its new name, twitch.

the key to starting a successful startup is to understand some group of users so well that you can make exactly what they want. if you're young you can, and should, use the hack of making something for yourself. you understand yourself. but this is just an instance of the more general rule. only by understanding users very deeply can you make something they love so much that they tell their friends about it, and only that can get you the exponential growth you need to make a startup really successful.

there are other ways to get rich than by starting startups. some of those do require you to exploit people. but startups are the most common way to become really rich, and if you want to start a successful startup, the key is not exploitation but empathy. what do users really want? what could you do for them that would make their lives dramatically better? that kind of empathy is what we look for in founders, and what we cultivate in the ones we accept.

how people become rich in your society is one of the most important things to understand about it. you can't let your beliefs about this be determined by ideology, or movies, or historical examples that are centuries old. you must look at the world around you and see how it's actually done. if you want to do it yourself, obviously you'll be forced to understand how it's done. so i don't worry too much about you. the ones i worry about are the future prime ministers. you need to remember this talk. so for you i'm going to summarize the key ideas.

there are two numbers that determine how big a startup gets, and thus how rich its founders become: the growth rate and how long it continues. you get the first by making something users like so much they tell their friends. you get the second by being in a big market. if you grow exponentially into a big market, your startup will become valuable, and you, as a shareholder, will become rich. you not only don't have to cheat to make this happen, it will happen automatically if you just keep making customers happy.

thanks to trevor blackwell, jared friedman, jessica livingston, and garry tan for reading drafts of this, and to arwa elrayess and the oxford union for hosting me.